

Functional Bootstrapping for a Single LWE Ciphertext with $\tilde{O}(1)$ Polynomial Multiplications

Xiaopeng Zheng^{1*}, Hongbo Li^{2,3}, and Dingkang Wang^{2,3}

¹ College of Mathematics and Computer Science, Shantou University, Shantou 515821, China

xiaopengzheng@stu.edu.cn

² State Key Laboratory of Mathematical Sciences, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing 100190, China

{hli,dwang}@mmrc.iss.ac.cn

³ School of Mathematical Sciences, University of Chinese Academy of Sciences, Beijing 100049, China

Abstract. Bootstrapping is the key technique that turns leveled homomorphic encryption into fully homomorphic encryption, but it remains a major efficiency bottleneck. Recent work by Z. Liu and Y. Wang (ASIACRYPT 2023) showed how to bootstrap N LWE ciphertexts with total cost of $\tilde{O}(N)$ polynomial multiplications based on the BFV scheme. However, their results achieve $\tilde{O}(1)$ complexity only through amortization over large batches, and do not give a genuine non-amortized $\tilde{O}(1)$ bound for a single ciphertext.

In this paper, we present a BFV-based functional bootstrapping algorithm for arbitrary functions over large plaintext spaces with total cost of $\tilde{O}(1)$ polynomial multiplications for one LWE ciphertext. The same construction also supports small and moderate batches, and processes a batch of m ciphertexts with total cost $\tilde{O}(m)$ in the supported parameter range. The main technical ingredient is a sparse-packing polynomial-evaluation method for BFV ciphertexts, which exploits the duplicated-slot structure to evaluate an arbitrary polynomial on m encrypted inputs with total cost of $\tilde{O}(m)$.

We implement the scheme in Lattigo using the BFV scheme. At 128-bit security and on a single thread, bootstrapping an arbitrary function takes 2.04 seconds for one ciphertext encrypting a 9-bit plaintext and 3.05 seconds for 128 such ciphertexts in one batched invocation. For 16-bit plaintexts, it takes 8.11 seconds for one ciphertext and 13.22 seconds for 16 ciphertexts. These results show that non-amortized single-ciphertext functional bootstrapping, as well as small and moderate batch bootstrapping, can be practical for arbitrary functions over relatively large plaintext spaces.

Keywords: Homomorphic encryption · Homomorphic polynomial evaluation · Functional bootstrapping · Large plaintext spaces

* Corresponding author.

1 Introduction

Fully homomorphic encryption (FHE) enables computation directly on encrypted data without requiring decryption, making it a central tool for privacy-preserving data processing, with applications such as private database queries and secure machine-learning inference [4,5,26,32]. Bootstrapping is the key operation that refreshes ciphertexts and thereby turns leveled homomorphic encryption into fully homomorphic encryption, but it remains one of the main efficiency bottlenecks in practice [22].

Since Gentry’s Ph.D. thesis introduced the first fully homomorphic encryption scheme and the bootstrapping technique [22], a number of more efficient leveled homomorphic encryption schemes have been developed, including BGV [9], BFV [19], and CKKS [13]. A major advantage of this line of work is that it supports Single Instruction Multiple Data (SIMD) operations, allowing many plaintext values to be packed into a single ciphertext and processed in parallel, which makes it efficient for evaluating circuits with many inputs. Recent years have witnessed substantial progress in bootstrapping. For BGV and BFV, bootstrapping evolved from early boolean-circuit approaches [23] to SIMD-based improvements [24], and then to a more algebraic line of work [2,11,21,31], with a further optimization of the polynomial step given in [20]. For CKKS, Cheon et al. introduced bootstrapping for approximate homomorphic encryption [12], and subsequent works further improved its efficiency and precision [7,10,38].

Another influential direction is the FHEW/TFHE line, which is rooted in the GSW approximate-eigenvector framework [25]. Ducas and Micciancio [18] introduced FHEW and showed that LWE ciphertexts can be bootstrapped in less than a second by using a ring-GSW accumulator. Chillotti, Gama, Georgieva, and Izabachène [14] subsequently developed TFHE, where the GSW internal product is replaced by an external product between GSW-type and LWE-type ciphertexts, leading to substantially faster gate bootstrapping.

In the FHEW/TFHE line, recent work has gone beyond the negacyclic setting and has significantly expanded the supported function class and plaintext precision [35,40,54,55,46,27,43,56,17]. Here, functional bootstrapping refers to the stronger bootstrapping task that, given an LWE ciphertext encrypting $\mu \in \mathbb{Z}_p$ and a target function $f : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$, outputs a fresh LWE ciphertext encrypting $f(\mu)$, with output noise small enough for correct decryption. In the batched case, the same operation is applied componentwise to many encrypted inputs. In parallel, batched and amortized bootstrapping has shown that many LWE ciphertexts can be processed together at very low amortized cost [47,48,41,42,29,44,28,3,1]. In particular, Liu and Wang [44] and Alexandru et al. [1] support arbitrary functional bootstrapping over relatively large plaintext spaces while achieving excellent amortized performance for very large batches.

Nevertheless, one important goal remains insufficiently addressed. Existing large-batch methods are designed to amortize their cost over thousands to tens of thousands of ciphertexts, and their guarantees do not directly translate into non-amortized $\tilde{O}(1)$ -cost functional bootstrapping for a single LWE ciphertext. For example, in the 12-bit setting, [44] and [1] report total running times of about

1280 seconds and 599.9 seconds to bootstrap 32768 and 131072 ciphertexts, respectively. These designs are compelling when the workload is large enough to fully exploit amortization, but they may perform substantial unnecessary computation when the server needs to refresh only one ciphertext or only a modest number of ciphertexts.

This paper asks whether one can obtain functional bootstrapping for arbitrary functions over large plaintext spaces with total cost $\tilde{O}(1)$ for a single ciphertext, while still remaining efficient for small and moderate batch sizes. We answer this question affirmatively. We propose a large-plaintext functional bootstrapping scheme for arbitrary functions that uses only $\tilde{O}(1)$ polynomial multiplications for one LWE ciphertext. More generally, for m ciphertexts, our method uses $\tilde{O}(m)$ polynomial multiplications and therefore retains amortized $\tilde{O}(1)$ complexity per ciphertext. We also implement the scheme and show that these asymptotic gains translate into practical performance.

The main bottleneck in the large-plaintext functional bootstrapping methods of Liu and Wang [44] and Alexandru et al. [1] is homomorphic polynomial evaluation. In both works, this step is carried out using the Paterson–Stockmeyer method. For a polynomial with dense coefficients of degree d , it requires d plaintext-ciphertext multiplications and $O(\sqrt{d})$ ciphertext-ciphertext multiplications. This cost is acceptable when amortized over very large batches, but it becomes a serious obstacle in the single-ciphertext and small-batch regimes.

Our main technical contribution is a new homomorphic polynomial evaluation algorithm for sparsely packed BGV ciphertexts. Suppose that a ciphertext sparsely encodes m messages in a ring of dimension N . Under the assumptions $d = O(N)$ and $d < (N/m)^2$, our method evaluates a degree- d polynomial on these m messages using only $\tilde{O}(m)$ basic homomorphic operations. More precisely, it uses $\tilde{O}(1)$ ciphertext-ciphertext multiplications, $O(md/N)$ plaintext-ciphertext multiplications, and $\tilde{O}(\sqrt{md/N})$ rotations. In particular, when $m = 1$, this yields $\tilde{O}(1)$ complexity for homomorphic polynomial evaluation on a single encrypted input. This sparse-packing polynomial-evaluation algorithm is the key ingredient that enables our non-amortized $\tilde{O}(1)$ functional bootstrapping result.

1.1 Our contributions

Homomorphic polynomial evaluation. In this paper, we consider the BGV scheme. Let $R_q = (\mathbb{Z}_q)[X]/(X^N + 1)$ denote the plaintext space, where q is the plaintext modulus and N is the ring dimension, assumed to be a power of two. Suppose that q is prime and satisfies $q \equiv 1 \pmod{2N}$. Then $X^N + 1$ splits completely over $\mathbb{Z}_q$, and hence $R_q \cong (\mathbb{Z}_q)^N$. Encoding a message as a vector in $(\mathbb{Z}_q)^N$ is called *slot encoding*.

For security considerations, the practical implementations of the BGV scheme typically use a large ring dimension, usually with $N \geq 4096$ [6]. On the other hand, in many applications, the message length is much smaller than the available slot capacity. In such cases, a natural approach is to replicate the message vector so as to fill all N slots. More precisely, if the original message vector has

length m , then it is repeated N/m times and encoded as

$$\underbrace{(x_0, \dots, x_{m-1}, x_0, \dots, x_{m-1}, \dots, x_0, \dots, x_{m-1})}_{N/m \text{ times}}.$$

We refer to this representation as *sparse packing*, since it corresponds to a polynomial in R_q with sparse coefficients. A similar sparse-packing strategy also appears in the CKKS bootstrapping literature [12].

Let $r = N/m$ denote the *repetition multiplicity*, namely, the number of repetitions in the sparse-packing representation. We study the following problem.

Problem 1 (Batched homomorphic polynomial evaluation for sparsely packed ciphertexts). Let ct be a BFV ciphertext encrypting $(x_0, x_1, \dots, x_{m-1})$ with repetition multiplicity r , and let $f_0, \dots, f_{m-1} \in \mathbb{Z}_q[x]$ be polynomials of degree $\leq d$. The goal is to efficiently compute a ciphertext encrypting

$$(f_0(x_0), f_1(x_1), \dots, f_{m-1}(x_{m-1}))$$

with the same repetition multiplicity r .

In this paper, we propose efficient algorithms for batched homomorphic polynomial evaluation when $d+1 \leq r^2$. In particular, we consider two cases, namely $d+1 \leq r$ and $r < d+1 \leq r^2$, and give an efficient algorithm for each case. For simplicity, we assume throughout that N , m , and $d+1$ are powers of two. Our main results are summarized as follows.

Under the conditions of Problem 1, one can homomorphically compute a ciphertext encrypting $(f_0(x_0), \dots, f_{m-1}(x_{m-1}))$, still with repetition multiplicity r , with the complexity shown in Table 1.

Table 1. Complexity comparison of the proposed homomorphic polynomial evaluation algorithms and the Paterson–Stockmeyer baseline

	Ours			PS baseline
Condition	$d+1 \leq r$	$r < d+1 < r^2$	$d+1 = r^2$	–
CMult	$\log(d+1)$	$\log(d+1) + \frac{m(d+1)}{N}$	$\log(d+1) + \frac{m(d+1)}{N}$	d
Mult	$2\log(d+1) - 2$	$2\log(d+1) - 2$	$2\log(d+1) - 2$	$O(\sqrt{d})$
Rotation	$\log(d+1)$	$\log(\frac{N}{m}) + 2\sqrt{\frac{m(d+1)}{N}} - 2$	$\log(\frac{N}{m}) + 4\sqrt{\frac{m(d+1)}{2N}} - 3$	0
Depth	$\log(d+1) + 1$	$\log(d+1) + 2$	$\log(d+1) + 2$	$\log(d+1)$

^a **CMult**: number of plaintext-ciphertext multiplications.

^b **Mult**: number of ciphertext-ciphertext multiplications.

^c **Rotation**: number of homomorphic rotations.

^d **Depth**: multiplicative depth.

^e In this paper, all logarithms are base 2 unless otherwise stated.

^f For the case $d+1 \leq r$, it incorporates the optimization in Remark 1, which merges the coefficient multiplication into the power-generation step.

Therefore, assuming $d < (N/m)^2$ and $d = O(N)$, we have $d/N = O(1)$. Hence, our method evaluates m polynomials using a total of $\tilde{O}(m)$ homomorphic operations, which gives amortized $\tilde{O}(1)$ complexity per polynomial.

Each homomorphic operation used in the circuit, namely ciphertext-ciphertext multiplication, plaintext-ciphertext multiplication, and homomorphic rotation, can be implemented using at most $\text{poly}(\log Q)$ polynomial multiplications over $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$. One polynomial multiplication means one multiplication of ring elements at modulus Q , which can be done with $O(N \log(N))\mathbb{Z}_Q$ operations using NTT. Hence, under the chosen leveled-BFV parameterization with $Q = \Theta(N^{\log N})$, each such operation costs at most $\tilde{O}(1)$ polynomial multiplications. Hence, in other words, our method evaluates m polynomials using a total of $\tilde{O}(m)$ polynomial multiplications in R_Q , which gives amortized $\tilde{O}(1)$ complexity per polynomial.

Large-Plaintext Functional Bootstrapping for Arbitrary Functions.

Building on the sparse polynomial-evaluation algorithm above, we obtain a functional bootstrapping scheme for arbitrary functions over large plaintext spaces. Assume throughout this subsection that q is prime, $q \equiv 1 \pmod{2N}$, $q = O(N)$, $N = O(\lambda)$, and $n = O(\lambda)$, and that the underlying LWE and RLWE assumptions hold. Let d be the smallest integer such that $d \geq q - 1$ and $d + 1$ is a power of two. The main result can be stated informally as follows.

Theorem 1 (Informal). *For every function $f : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$, one can bootstrap a single LWE ciphertext encrypting $\mu \in \mathbb{Z}_p$ to a refreshed LWE ciphertext encrypting $f(\mu)$ using only $\tilde{O}(1)$ polynomial multiplications in $R_Q = \mathbb{Z}_Q[X]/(X^N + 1)$. More generally, for every $m \leq N/\sqrt{d} + 1$, one can bootstrap m LWE ciphertexts for arbitrary functions using a total of $\tilde{O}(m)$ polynomial multiplications in R_Q .*

Implementation. We implement our general functional bootstrapping scheme in the homomorphic encryption library Lattigo [36] under a range of parameter settings and compare it with prior work. For each setting, we test two cases for the number m of input LWE ciphertexts: $m = 1$ and the largest value of m supported by that setting. We consider plaintext moduli corresponding to message precisions of 9, 12, 14, and 16 bits, and all parameter sets provide 128-bit security. The experimental results are reported in Table 2.

The experimental results show that our implementation supports bootstrapping over large plaintext spaces and achieves substantially smaller running time per invocation than existing batch bootstrapping methods. Our goal is not to improve the best amortized throughput in the very large-batch regime, but rather to reduce total latency in the single-ciphertext, small-batch, and moderate-batch regimes.

1.2 Related Works

Homomorphic polynomial evaluation. Our work is related to homomorphic polynomial evaluation. The classical Paterson–Stockmeyer method [51] evaluates a degree- d polynomial with $O(\sqrt{d})$ nonscalar multiplications. Okada, Player, and

Table 2. Comparison for functional batch bootstrapping for various input bit-sizes

Scheme(s)	Number of input LWE ciphertexts	Number of input/output bits	Total time	Amortized time
[29]	1024	8	871.8s	0.851s
[28]	8192	8	232s	0.0285s
[44]	32768	9	220s	0.0067s
	32768	12	1280s	0.0391s
[3]	4096	8	15.4s	0.0038s
	4096	10	50.3s	0.0123s
[1]	65536	9	63.96s	0.0012s
	131072	12	599.9s	0.0046s
	131072	14	2130.1s	0.0162s
This work	1	9	2.040s	2.040s
	128	9	3.054s	0.0239s
	1	12	6.198s	6.198s
	64	12	9.733s	0.1521s
	1	14	7.715s	7.715s
	32	14	13.711s	0.4285s
	1	16	8.113s	8.113s
	16	16	13.220s	0.8263s

* The results for prior works are taken directly from the corresponding papers, while the results for this work are obtained from our updated implementation. These comparisons are cross-implementation and cross-platform, and are intended to illustrate the regime in which our method is advantageous, rather than provide a cycle-accurate apples-to-apples benchmark.

Pohmann [49] exploited Galois structure to reduce the multiplicative cost for certain packed polynomial-evaluation settings and applied the method to BFV bootstrapping. More recently, Wang and Wang [53] proposed an asymmetric BSGS algorithm for arbitrary dense polynomials in leveled homomorphic encryption, reducing the number of modulus and key switches. Park et al. [50] proposed a slim homomorphic polynomial evaluation algorithm for real-coefficient polynomials, based on a recursive sum-of-squares decomposition $P(x) = U(x)^2 + V(x)^2 - m$, which exploits sparsely packed CKKS ciphertexts and evaluates degree- d polynomials using only $O(\log d)$ homomorphic operations.

Functional bootstrapping and its extensions to larger precision in the TFHE line. Programmable or functional bootstrapping is a central tool in the TFHE setting. Chillotti, Joye, and Paillier [15] showed how programmable bootstrapping can be used for efficient homomorphic inference of deep neural networks. Chillotti et al. [16] then improved programmable bootstrapping in TFHE, supporting larger precision and more efficient arithmetic circuits. Liu, Micciancio, and Polyakov [43] further studied large-precision sign evaluation in

the FHEW/TFHE setting. Later works extended functional bootstrapping beyond the basic small-domain setting in several closely related directions. Klucznik and Schild [35] initiated the full-domain line by removing the negacyclic restriction and supporting arbitrary lookup tables. Li et al. [40], Xia, Liu, and Wang [55], and Ma et al. [46] further improved full-domain and general-function bootstrapping, with Ma et al. [46] also improving homomorphic digit decomposition. Wang et al. [54] studied full-domain functional bootstrapping in the prime-cyclotomic setting. Xu et al. [56] and Duan et al. [17] developed large-plaintext variants with improved parameter efficiency and smaller bootstrapping keys. Overall, these works substantially broaden the supported function class and plaintext space, but they do not focus on the small-batch SIMD amortization setting considered in this work.

Batched and amortized bootstrapping. Batched and amortized bootstrapping have also been studied extensively. Micciancio and Sorrell [47] introduced amortized bootstrapping for FHEW-like ciphertexts via ring packing. De Micheli, Kim, Micciancio, and Suhl [48] further improved this direction using ring automorphisms. Liu and Wang [41] introduced a new framework for SIMD bootstrapping in polynomial modulus and achieved amortized $\tilde{O}(\lambda^{0.75})$ FHE multiplications and their sequel [42] improved this to amortized $\tilde{O}(1)$ FHE multiplications. Guimarães, Pereira, and van Leeuwen [29] later revisited this line with a simpler and implemented construction. More recently, Guimarães and Pereira [28] proposed a fast amortized bootstrapping construction with small keys and polynomial noise overhead. For functional bootstrapping, Liu and Wang [44] showed that batched arbitrary functional bootstrapping can be achieved with amortized $\tilde{O}(1)$ polynomial multiplications, while Lee, Min, and Song [37] proposed functional bootstrapping for packed ciphertexts via homomorphic LUT evaluation, supporting both FV-to-FV and CKKS-to-FV settings. These works greatly reduce the amortized cost, but they are mainly designed for large batch sizes. Among them, the closest to our work is [44]. Our construction follows the same general direction, but replaces the dense polynomial-evaluation step by a sparse-packing algorithm that is better suited to small and moderate batch sizes.

Recent CKKS-based directions. Recent works also showed that CKKS can support richer discrete computations than standard approximate arithmetic. Alexandru, Kim, and Polyakov [1] proposed general functional bootstrapping using CKKS, based on trigonometric Hermite interpolation, and obtained very low amortized cost for large batches. Bae, Kim, Stehlé, and Suvanto [3] proposed CKKS bootstrapping for small integers and bits: their SI-BTS supports small-integer functional bootstrapping, while their BB-BTS reduces the amortized cost of bit bootstrapping.

To make the comparison more explicit, Table 3 summarizes the most relevant prior works in terms of total cost, amortized cost, supported functionality, and implementation status.

Table 3. Comparison with prior work on batched LWE ciphertext bootstrapping in terms of asymptotic homomorphic operation counts

Scheme(s)	Total cost	Messages	Amortized cost	Arbitrary function	Implementation
Ours (single-ciphertext)	$\tilde{O}(1)$	1	$\tilde{O}(1)$	Yes	Yes
Ours (maximal supported batch)	$\tilde{O}(\sqrt{n})$	$O(\sqrt{n})$	$\tilde{O}(1)$	Yes	Yes
[18,14,39]	$O(n)$	1	$O(n)$	No	No
[47]	$\tilde{O}(3^{1/\epsilon} \cdot n^{1+\epsilon})$	$O(n)$	$\tilde{O}(3^{1/\epsilon} \cdot n^\epsilon)$	No	No
[29,48]	$O(\epsilon \cdot n^{1+1/\epsilon})$	$O(n)$	$O(\epsilon \cdot n^{1/\epsilon})$	Yes	Yes (by [29])
[28]	$\tilde{O}(nh)$	$O(n)$	$\tilde{O}(h)$	Yes	Yes
[42]	$\tilde{O}(n)$	$O(n)$	$\tilde{O}(1)$	No	No
[44]	$\tilde{O}(n)$	$O(n)$	$\tilde{O}(1)$	Yes	Yes
[3]	$O(n^{1/\ell}) + O(p)$	$O(n)$	$O(1)$	Yes	Yes
[1]	$O(n^{1/\ell}) + O(p)$	$O(n)$	$O(1)$	Yes	Yes

^a Suppose that $n = O(\lambda)$, where n denotes the LWE ciphertext dimension and λ is the security parameter. If RLWE ciphertexts are used in the scheme, we further assume that $N = O(\lambda)$ and $N = O(n)$, where N denotes the RLWE ciphertext dimension.

^b In our scheme, we assume that $q > 2N$ and $q = O(N)$, where q denotes the modulus. This is the same assumption as in [44]. Under these assumptions, the condition $m \leq N/\sqrt{q}$ implies $m = O(\sqrt{n})$.

^c In [42], it is assumed that $q = O(\sqrt{N})$.

^d For [3] and [1], the total cost consists of two parts. The term $O(n^{1/\ell})$ comes from the CoeffToSlot and SlotToCoeff procedures, where ℓ denotes the number of layers in the FFT decomposition. The term $O(p)$ comes from homomorphic polynomial evaluation, where p is the plaintext modulus of the input LWE ciphertext. In the amortized complexity, we assume $p = O(n)$.

^e For [28], h denotes the Hamming weight of the sparse LWE/RLWE secret. The displayed bound uses the gap bound $B = O(n/h)$ for the sparse-polynomial multiplication step.

1.3 Our Techniques

Homomorphic polynomial evaluation. We propose efficient algorithms for batched homomorphic polynomial evaluation when $d + 1 \leq r^2$. For simplicity, we explain the main ideas in the plaintext setting. All the operations involved can be performed homomorphically in the BFV scheme. We consider two cases: $d + 1 \leq r$ and $r < d + 1 \leq r^2$. Our main techniques are illustrated by two simple examples with $m = 1$.

Let $N = 8$ and $m = 1$. Then the repetition multiplicity is $r = 8$, and a single message is encoded in $\mathbb{Z}_q^8$ as $\mathbf{x} = (x, x, x, x, x, x, x, x)$. We aim to evaluate

$$f(x) = a_0 + a_1x + a_2x^2 + a_3x^3 + a_4x^4 + a_5x^5 + a_6x^6 + a_7x^7.$$

Let $\mathbf{f} = (a_0, a_1, a_2, a_3, a_4, a_5, a_6, a_7)$. Let $\mathbf{x}_0 = \mathbf{x}$. First, compute

$$\begin{aligned} \mathbf{x}_1 &= \mathbf{x}_0 \odot \mathbf{x}_0 = (x^2, x^2, x^2, x^2, x^2, x^2, x^2, x^2), \\ \mathbf{x}_2 &= \mathbf{x}_1 \odot \mathbf{x}_1 = (x^4, x^4, x^4, x^4, x^4, x^4, x^4, x^4). \end{aligned} \tag{1}$$

where $\odot$ denotes the Hadamard product (componentwise multiplication). Let

$$C = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix} = \begin{pmatrix} \mathbf{c}_0 \\ \mathbf{c}_1 \\ \mathbf{c}_2 \end{pmatrix} \text{ and } D = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} \mathbf{d}_0 \\ \mathbf{d}_1 \\ \mathbf{d}_2 \end{pmatrix}.$$

where the j -th column of C is the binary expansion of $j-1$ and D is the entrywise complement of C . Let

$$\begin{aligned} \mathbf{y}_0 &= \mathbf{x}_0 \odot (\mathbf{c}_0 \odot \mathbf{f}) + \mathbf{d}_0 \odot \mathbf{f} = (a_0, a_1x, a_2, a_3x, a_4, a_5x, a_6, a_7x), \\ \mathbf{y}_1 &= \mathbf{x}_1 \odot \mathbf{c}_1 + \mathbf{d}_1 = (1, 1, x^2, x^2, 1, 1, x^2, x^2), \\ \mathbf{y}_2 &= \mathbf{x}_2 \odot \mathbf{c}_2 + \mathbf{d}_2 = (1, 1, 1, 1, x^4, x^4, x^4, x^4), \end{aligned} \quad (2)$$

Then we obtain

$$\mathbf{y} = \mathbf{y}_0 \odot \mathbf{y}_1 \odot \mathbf{y}_2 = (a_0, a_1x, a_2x^2, a_3x^3, a_4x^4, a_5x^5, a_6x^6, a_7x^7). \quad (3)$$

We use the standard rotate-and-add technique to sum up all slots and obtain

$$\mathbf{h} = (f(x), f(x), f(x), f(x), f(x), f(x), f(x), f(x)).$$

When $d+1 > r$, we can no longer pack all powers $1, x, \dots, x^d$ into the r slots as in the above example. Therefore, the above procedure must be modified. For example, let $N = 8$, $m = 1$ and $d = 31$. Then $r = N/m = 8$. We have $d+1 \leq r^2$. Suppose that we want to evaluate

$$f(x) = a_0 + a_1x + \dots + a_{31}x^{31}.$$

Similarly, we compute

$$\mathbf{x}_i = (x^{2^i}, \dots, x^{2^i}), \text{ for } i = 1, \dots, 4, \quad (4)$$

where $4 = \log(d+1) - 1$. Applying the same construction as in (2), and (3), with the coefficient vector $\mathbf{f}$ replaced by $(1, 1, 1, 1, 1, 1, 1, 1)$, we obtain

$$\mathbf{p} = (1, x, x^2, x^3, x^4, x^5, x^6, x^7). \quad (5)$$

Consider a matrix

$$F = \begin{pmatrix} a_0 & a_1 & a_2 & a_3 & a_4 & a_5 & a_6 & a_7 \\ a_8 & a_9 & a_{10} & a_{11} & a_{12} & a_{13} & a_{14} & a_{15} \\ a_{16} & a_{17} & a_{18} & a_{19} & a_{20} & a_{21} & a_{22} & a_{23} \\ a_{24} & a_{25} & a_{26} & a_{27} & a_{28} & a_{29} & a_{30} & a_{31} \end{pmatrix} = \begin{pmatrix} \mathbf{f}_0 \\ \mathbf{f}_1 \\ \mathbf{f}_2 \\ \mathbf{f}_3 \end{pmatrix}$$

Perform a linear transformation of $\mathbf{p}$ by F and obtain

$$\mathbf{h}^{(0)} = (h_0, h_1, h_2, h_3, h_0, h_1, h_2, h_3), \text{ where } h_j = \langle \mathbf{f}_j, \mathbf{p} \rangle = \sum_{i=0}^7 a_{8j+i} x^i.$$

Applying the same construction as in (2) and (3) requires the vectors

$$\mathbf{x}_3 = (x^8, \dots, x^8) \quad \text{and} \quad \mathbf{x}_4 = (x^{16}, \dots, x^{16}).$$

We also adjust the columns of C so that they correspond to the bit decompositions of “0,1,2,3,0,1,2,3”, respectively. From this, we obtain

$$\mathbf{g} = (1, x^8, x^{16}, x^{24}, 1, x^8, x^{16}, x^{24}).$$

Then $\mathbf{g} \odot \mathbf{h}^{(0)} = (g_1, g_2, g_3, g_4, g_1, g_2, g_3, g_4)$, where

$$g_j = \sum_{i=0}^7 a_{8(j-1)+i} x^{i+8(j-1)}, \quad 1 \leq j \leq 4.$$

Finally, using the standard rotate-and-add technique, we sum g_1, g_2, g_3, g_4 and obtain

$$\mathbf{h} = (f(x), f(x), f(x), f(x), f(x), f(x), f(x), f(x)).$$

Large-Plaintext Functional Bootstrapping for Arbitrary Functions.

Our construction follows the basic idea of [44], with several modifications. We illustrate the method by a toy example in the case $m = 1$.

Let $\mathbf{c} = (\mathbf{a}, b) \in \mathbb{Z}_{17}^5$ be an LWE ciphertext satisfying

$$b = \langle \mathbf{a}, \mathbf{s} \rangle + \alpha\mu + e \pmod{q},$$

where $\mu \in \mathbb{Z}_2$ is the message, $\mathbf{s} \in \mathbb{Z}^4$ is the secret key, $\alpha = \lfloor 17/2 \rfloor = 8$, and $|e| < \lfloor 17/(2 \cdot 2) \rfloor = 4$. Suppose that we use a BFV scheme with ring dimension $N = 8$ and plaintext modulus $q = 17$, which is equal to the ciphertext modulus of LWE ciphertext. The corresponding bootstrapping key is a BFV ciphertext $\mathbf{ct}$ encrypting $(s_0, s_0, s_1, s_1, s_2, s_2, s_3, s_3)$, where $(s_0, s_1, s_2, s_3) = \mathbf{s}$.

In the bootstrapping procedure, we first perform a plaintext-ciphertext multiplication between $\mathbf{ct}$ and the plaintext vector $-(a_0, a_0, a_1, a_1, a_2, a_2, a_3, a_3)$, and obtain a ciphertext encrypting

$$-(a_0 s_0, a_0 s_0, a_1 s_1, a_1 s_1, a_2 s_2, a_2 s_2, a_3 s_3, a_3 s_3).$$

Next, by the standard rotate-and-add technique, we sum the terms $a_0 s_0, a_1 s_1, a_2 s_2, a_3 s_3$ and obtain a ciphertext encrypting

$$-(\langle \mathbf{a}, \mathbf{s} \rangle, \langle \mathbf{a}, \mathbf{s} \rangle, \langle \mathbf{a}, \mathbf{s} \rangle, \langle \mathbf{a}, \mathbf{s} \rangle, \langle \mathbf{a}, \mathbf{s} \rangle, \langle \mathbf{a}, \mathbf{s} \rangle, \langle \mathbf{a}, \mathbf{s} \rangle, \langle \mathbf{a}, \mathbf{s} \rangle).$$

After adding the plaintext vector $(b, b, \dots, b)$, we obtain a ciphertext $\mathbf{ct}_1$ encrypting

$$(\alpha\mu + e, \alpha\mu + e, \alpha\mu + e, \alpha\mu + e, \alpha\mu + e, \alpha\mu + e, \alpha\mu + e, \alpha\mu + e).$$

Now let $g : \mathbb{Z}_2 \rightarrow \mathbb{Z}_2$ be an arbitrary function. Since $|e| < 4$, there exists a polynomial f of degree 16 such that $f(\alpha\mu + e) = \alpha g(\mu)$. The construction of f is given in [44], and we will briefly review it in Section 5. We then apply our homomorphic polynomial evaluation algorithm to $\mathbf{ct}_1$ and obtain a ciphertext $\mathbf{ct}_2$ encrypting

$$(\alpha g(\mu), \alpha g(\mu), \alpha g(\mu), \alpha g(\mu), \alpha g(\mu), \alpha g(\mu), \alpha g(\mu), \alpha g(\mu)).$$

In this special case, all slots of $\mathbf{ct}_2$ are equal, so $\mathbf{ct}_2$ already encrypts the constant polynomial $\alpha g(\mu)$, and no SlotToCoeff step is needed. Finally, by modulus switching and key switching, we convert this BFV ciphertext into an LWE ciphertext, using the same extraction technique as in [44]. By properly choosing the BFV parameters, we ensure that the resulting LWE ciphertext encrypts $\alpha g(\mu)$ with new noise e' satisfying $|e'| < 4$.

1.4 Organization of the paper

The rest of the paper is organized as follows. Section 2 reviews the preliminaries on LWE, BFV, and homomorphic linear transformations. Section 3 presents the basic algorithms used in our construction, including batched linear transformation and batched power generation. Section 4 develops our homomorphic polynomial evaluation algorithms for sparsely packed ciphertexts in the two cases $d + 1 \leq r$ and $r < d + 1 \leq r^2$. Section 5 applies these techniques to functional bootstrapping and describes the three main steps of the construction. Section 6 reports the implementation details, parameter choices, and experimental results. Finally, Section 7 concludes the paper.

2 Preliminaries

2.1 LWE problem and LWE ciphertext

For an integer $q \geq 2$, let $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$. We first recall the Learning With Errors (LWE) problem introduced by Regev [52].

Let $n \geq 1$ be the dimension, let $q \geq 2$ be the modulus, let χ_s be a distribution over $\mathbb{Z}_q^n$, and let χ_e be an error distribution over $\mathbb{Z}$. For a secret vector $\mathbf{s} \leftarrow \chi_s$, define the LWE distribution $A_{\mathbf{s}, \chi_e}$ over $\mathbb{Z}_q^n \times \mathbb{Z}_q$ as follows: sample $\mathbf{a} \leftarrow \mathbb{Z}_q^n$ uniformly at random, sample $e \leftarrow \chi_e$, and set

$$b = \langle \mathbf{a}, \mathbf{s} \rangle + e \pmod{q}.$$

The output is the pair $(\mathbf{a}, b)$.

The decisional LWE problem, denoted by $\text{LWE}_{n, q, \chi_s, \chi_e}$, is to distinguish samples drawn from $A_{\mathbf{s}, \chi_e}$ from samples drawn uniformly at random from $\mathbb{Z}_q^n \times \mathbb{Z}_q$. When needed, we also consider the search version of LWE, whose goal is to recover the secret $\mathbf{s}$ from sufficiently many samples of $A_{\mathbf{s}, \chi_e}$. Throughout this work, we assume that the decisional LWE problem $\text{LWE}_{n, q, \chi_s, \chi_e}$ is computationally hard for the parameter sets under consideration, and we refer to this hardness assumption as the LWE assumption.

LWE ciphertexts. We now introduce the LWE encryption scheme used in this work. Let $p \geq 2$ be the plaintext modulus and let $q \geq 2$ be the ciphertext modulus. Define $\Delta = \lfloor q/p \rfloor$. Under a secret key $\mathbf{s} \leftarrow \chi_s \subseteq \mathbb{Z}_q^n$, an LWE encryption of a message $m \in \mathbb{Z}_p$ is a pair $\text{ct} = (\mathbf{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$, where $\mathbf{a} \leftarrow \mathbb{Z}_q^n$ is sampled uniformly at random, $e \leftarrow \chi_e$ satisfies $|e| < \lfloor q/(2p) \rfloor$, and

$$b = \langle \mathbf{a}, \mathbf{s} \rangle + \Delta m + e \pmod{q}.$$

To decrypt ct , compute $v = b - \langle \mathbf{a}, \mathbf{s} \rangle \pmod{q}$, so that ideally $v = \Delta m + e$. The plaintext is then recovered by rounding pv/q to the nearest element of $\mathbb{Z}_p$. In particular, decryption is correct whenever the error satisfies $|e| < \lfloor q/(2p) \rfloor$.

Modulus switching is a standard operation that changes the ciphertext modulus while preserving the encrypted plaintext. Let $\text{ct} = (\mathbf{a}, b) \in \mathbb{Z}_q^{n+1}$ be an LWE

ciphertext encrypting $m \in \mathbb{Z}_p$ under secret key $\mathbf{s}$, with ciphertext modulus q and error bounded by β . For a target modulus q' , the modulus-switching procedure is defined by scaling and rounding each component:

$$\mathbf{ct}' = (\mathbf{a}', b') \leftarrow \text{ModSwitch}_{q \rightarrow q'}(\mathbf{ct}) := \left\lfloor \frac{q'}{q} \mathbf{ct} \right\rfloor = \left(\left\lfloor \frac{q'}{q} \mathbf{a} \right\rfloor, \left\lfloor \frac{q'}{q} b \right\rfloor \right) \in \mathbb{Z}_{q'}^{n+1}.$$

Then $\mathbf{ct}'$ encrypts the same message m under the same secret key $\mathbf{s}$, but with ciphertext modulus q' . More precisely, writing $\Delta' = \lfloor q'/p \rfloor$, the decryption error after modulus switching satisfies

$$|b' - \langle \mathbf{a}', \mathbf{s} \rangle - \Delta' m| < \frac{q'}{q} \beta + \sigma_{\text{ms}},$$

where σ_{ms} denotes the additional rounding error introduced by modulus switching. In practice, when the input ciphertext is sufficiently random, or when randomized rounding is used, this worst-case bound can be replaced by a smaller probabilistic bound of order $O(\|\mathbf{s}\|_2)$. For uniformly random ternary secret keys, this term is often estimated as $\sigma_{\text{ms}} \approx \sqrt{\frac{\|\mathbf{s}\|_2^2 + 1}{12}}$. We use this modulus-switching bound in the subsequent noise analysis, following the formulation of [18].

Key switching is also a standard operation that changes the secret key while preserving the encrypted plaintext. Concretely, given an LWE ciphertext $\mathbf{ct} \in \mathbb{Z}_q^{n+1}$ encrypting a message $m \in \mathbb{Z}_p$ under secret key $\mathbf{s} \in \mathbb{Z}_q^n$, and a key-switching key K from $\mathbf{s}$ to $\mathbf{s}' \in \mathbb{Z}_{q'}^n$, the procedure $\text{KeySwitch}_K(\mathbf{ct})$ outputs a ciphertext in $\mathbb{Z}_{q'}^{n+1}$ that encrypts the same message m under the new secret key $\mathbf{s}'$. The ciphertext modulus and plaintext remain unchanged, while the error increases by a controlled amount determined by the decomposition parameters and the noise in K . We also use key switching as a black box and refer the reader to [18] for details.

2.2 BFV Leveled Homomorphic Encryption

We briefly recall the Brakerski/Fan-Vercauteren (BFV) leveled homomorphic encryption scheme. BFV was first introduced in the standard-LWE setting and was later adapted to the ring-LWE setting [8, 19].

Let N be a power of two, and define $R = \mathbb{Z}[X]/(X^N + 1)$, $R_M = R/MR$ for any positive integer M . We write q for the plaintext modulus and Q for the ciphertext modulus, where $q < Q$. Throughout this paper, we assume that q is prime and satisfies $q \equiv 1 \pmod{2N}$. Under this assumption, $X^N + 1$ splits completely over $\mathbb{Z}_q$, and the plaintext ring R_q admits a slot decomposition $R_q \cong \mathbb{Z}_q^N$. Hence, a vector $\mathbf{m} = (m_0, \dots, m_{N-1}) \in \mathbb{Z}_q^N$ can be encoded into a polynomial $m(X) \in R_q$ by an inverse number theoretic transform (INTT), and decoded back by the corresponding number theoretic transform (NTT). This gives the usual SIMD functionality of BFV [30].

We use Enc and Dec to denote the encryption and decryption algorithms, respectively. For simplicity, when we say that a ciphertext $\mathbf{ct}$ encrypts a vector

$\mathbf{m}$, we mean that ct is a BFV ciphertext encrypting the polynomial encoded from $\mathbf{m}$.

Let $\Delta = \lfloor Q/q \rfloor$. Let $s \leftarrow \chi_{\text{key}} \subset R$ be a secret polynomial, and let χ_{err} be an error distribution over R . A symmetric BFV encryption of a plaintext $m(X) \in R_q$ is a pair $\text{ct} = (a, b) \in R_Q^2$, where $a \leftarrow R_Q$ is sampled uniformly at random, $e \leftarrow \chi_{\text{err}}$, and

$$b = a \cdot s + \Delta m(X) + e \pmod{Q}.$$

Decryption computes

$$\mu(X) = \left\lfloor \frac{q}{Q} (b - a \cdot s \pmod{Q}) \right\rfloor \in R_q,$$

and then decodes $\mu(X)$ to recover the underlying message vector $\mathbf{m}$. In particular, decryption is correct whenever the coefficients of the error term remain sufficiently smaller than $Q/(2q)$.

Homomorphic operators. The BFV scheme supports the following operators:

Add, CMult, Mult, ColumnRotation_{*i*}, RowRotation, KeySwitch, ModSwitch _{$Q \rightarrow Q'$} .

Here Add, CMult, Mult, KeySwitch, and ModSwitch are standard BFV operators, while ColumnRotation_{*i*} and RowRotation denote the two types of slot rotations under the $2 \times (N/2)$ hypercube view introduced below.

Add. For two ciphertexts ct_1, ct_2 encrypting $\mathbf{m}_1, \mathbf{m}_2 \in \mathbb{Z}_q^N$, we write $\text{Add}(\text{ct}_1, \text{ct}_2)$ for the ciphertext-ciphertext addition. Therefore, $\text{Dec}(\text{Add}(\text{ct}_1, \text{ct}_2)) = \mathbf{m}_1 + \mathbf{m}_2$, whenever the resulting ciphertext remains decryptable. Let $\mathbf{u} \in \mathbb{Z}_q^N$ be a plaintext vector, we write $\text{Add}(\text{ct}, \mathbf{u})$ for the plaintext-ciphertext addition by the encoded polynomial corresponding to $\mathbf{u}$. Then $\text{Dec}(\text{Add}(\text{ct}, \mathbf{u})) = \mathbf{m} + \mathbf{u}$.

CMult. Let ct be a BFV ciphertext encrypting a plaintext vector $\mathbf{m} \in \mathbb{Z}_q^N$, and let $\mathbf{u} \in \mathbb{Z}_q^N$ be a plaintext vector. We write $\text{CMult}(\text{ct}, \mathbf{u})$ for the plaintext-ciphertext multiplication by the encoded polynomial corresponding to $\mathbf{u}$. Then $\text{Dec}(\text{CMult}(\text{ct}, \mathbf{u})) = \mathbf{u} \odot \mathbf{m}$, where $\odot$ denotes the Hadamard product over $\mathbb{Z}_q^N$.

Mult. For two ciphertexts ct_1, ct_2 encrypting $\mathbf{m}_1, \mathbf{m}_2 \in \mathbb{Z}_q^N$, we write $\text{Mult}(\text{ct}_1, \text{ct}_2)$ for the ciphertext-ciphertext multiplication. Concretely, this operation first forms the tensor-product ciphertext with three polynomial components, and then applies relinearization using an evaluation key so that the output again consists of two ring elements. Therefore, $\text{Dec}(\text{Mult}(\text{ct}_1, \text{ct}_2)) = \mathbf{m}_1 \odot \mathbf{m}_2$, whenever the resulting ciphertext remains decryptable.

ColumnRotation and RowRotation. Consider the vector

$$\mathbf{m} = (a_0, a_1, \dots, a_{N/2-1}, b_0, b_1, \dots, b_{N/2-1}) \in \mathbb{Z}_q^N,$$

where the indices below are taken modulo $N/2$. We view $\mathbf{m}$ as a $2 \times (N/2)$ hypercube. Define

$$\begin{aligned} \rho_i(\mathbf{m}) &= (a_i, a_{i+1}, \dots, a_{i-1}, b_i, b_{i+1}, \dots, b_{i-1}), \quad i = 0, 1, \dots, N/2 - 1, \\ \tau(\mathbf{m}) &= (b_0, b_1, \dots, b_{N/2-1}, a_0, a_1, \dots, a_{N/2-1}). \end{aligned}$$

We write $\text{ColumnRotation}_i(\text{ct})$ for the ciphertext obtained by rotating each row by i positions, and $\text{RowRotation}(\text{ct})$ for the ciphertext obtained by swapping the two rows. Namely, $\text{Dec}(\text{ColumnRotation}_i(\text{ct})) = \rho_i(\mathbf{m})$, $\text{Dec}(\text{RowRotation}(\text{ct})) = \tau(\mathbf{m})$. These operations are realized by applying suitable Galois automorphisms to the ciphertext components, followed by key switching back to the original secret key using the corresponding rotation keys.

KeySwitch. We write $\text{KeySwitch}_{s \rightarrow s'}(\text{ct})$ for the standard BGV key-switching procedure, which transforms a ciphertext under secret key s into a ciphertext under another secret key s' , while preserving the encrypted plaintext. In particular, both relinearization and slot rotation are implemented using suitable instances of key switching.

ModSwitch. For two ciphertext moduli $Q > Q'$, we write $\text{ModSwitch}_{Q \rightarrow Q'}(\text{ct})$ for the modulus-switching procedure. If $\text{ct} = (a, b) \in R_Q^2$, then $\text{ModSwitch}_{Q \rightarrow Q'}(\text{ct})$ rescales each coefficient of a and b from modulus Q to modulus Q' and rounds to the nearest integer. This operation preserves the encrypted plaintext while changing the ciphertext modulus, and is mainly used to control noise growth.

Since our constructions use these BGV procedures as black boxes, we omit further low-level details and refer the reader to [19,30,33].

2.3 Linear Transformation

A homomorphic linear transformation (HLT), introduced in [30], has become a standard primitive in homomorphic machine learning. In their work, Halevi and Shoup proposed a diagonal method for multiplying a length- r vector $\mathbf{m}$ by an $r \times r$ matrix A .

Subsequently, PEGASUS [45] generalized homomorphic linear transformations to the rectangular setting. Given a matrix $A \in \mathbb{Z}_q^{\ell \times r}$, its method conceptually tiles A into a square structure, then extracts the required diagonal vectors, and then applies a BSGS strategy to evaluate the transformation efficiently. The algorithm explicitly distinguishes between the tall case $\ell \geq r$ and the short case $\ell < r$. As the short case will play an important role in Section 3.1, we present the corresponding procedure in detail here.

For the short case $\ell < r$, let $A = (a_{i,k}) \in \mathbb{Z}_q^{\ell \times r}$ and $\mathbf{m} \in \mathbb{Z}_q^r$. Assume for simplicity that $\ell \mid r$. To apply the diagonal method, we first conceptually extend

A to an $r \times r$ square matrix $\tilde{A}$ by repeating its rows cyclically, namely

$$\tilde{A}_{i,k} = a_{(i \bmod \ell), k}, \quad 0 \leq i, k < r.$$

For each $0 \leq j < \ell$, we then extract the j -th diagonal vector $\mathbf{u}_j$ of $\tilde{A}$, that is,

$$\mathbf{u}_j = (a_{0,j}, a_{1,j+1}, \dots, a_{\ell-1,j+\ell-1}, a_{0,j+\ell}, a_{1,j+\ell+1}, \dots, a_{\ell-1,j+r-1}) \in \mathbb{Z}_q^r,$$

where the second index is taken modulo r . For convenience, we shall refer to $\mathbf{u}_j$ as the j -th diagonal of A throughout this paper. Using these diagonal vectors, we first form the intermediate vector

$$\mathbf{y} = \sum_{0 \leq j < \ell} (\mathbf{u}_j \odot \rho_j(\mathbf{m})). \quad (6)$$

Unlike the square case, $\mathbf{y}$ is not yet equal to $\mathbf{m}A^T$. Instead, one can recover $\mathbf{m}A^T$ from $\mathbf{y}$ by a binary rotation-and-sum procedure: set $\mathbf{y}^{(0)} = \mathbf{y}$, and for $0 \leq i < \gamma$, where $\gamma = \log(r/\ell)$, define

$$\mathbf{y}^{(i+1)} = \mathbf{y}^{(i)} + \rho_{2^i \ell}(\mathbf{y}^{(i)}).$$

After γ steps, the vector $\mathbf{y}^{(\gamma)}$ becomes an ℓ -periodic repetition of $\mathbf{m}A^T$, namely

$$\mathbf{y}^{(\gamma)} = (\mathbf{m}A^T \parallel \dots \parallel \mathbf{m}A^T),$$

Moreover, the computation of (6) can be accelerated by the baby-step giant-step (BSGS) technique. More precisely, if $\ell = bg$, then the intermediate vector $\mathbf{y}$ can be computed using $b + g - 2$ rotations and ℓ Hadamard products. Consequently, together with the additional $\log(r/\ell)$ rotation-and-sum steps, the total number of rotations in the short case is $b + g - 2 + \log(r/\ell)$, while the number of plaintext-ciphertext multiplications remains ℓ .

2.4 Notations

Throughout the paper, we use the notation summarized in Table 4. In particular, we set the BFV plaintext modulus equal to the LWE ciphertext modulus and denote both by q . Unless otherwise stated, all logarithms are base 2. We also assume throughout that N , m , r , and $d+1$ are powers of two, that $N = rm$, and that $q \equiv 1 \pmod{2N}$. Under this condition, the BFV plaintext ring admits the usual slot decomposition, and the hypercube rotations introduced in Section 2.2 are available.

Table 4. Core notation used throughout the paper

Symbol	Meaning	Symbol	Meaning
n	LWE dimension	p	LWE plaintext modulus
N	BFV ring dimension	q	common modulus ^a
m	number of input ciphertexts / packed messages	Q	BFV ciphertext modulus
r	repetition multiplicity, $r = N/m$	d	polynomial degree bound

^a Throughout the paper, q denotes both the LWE ciphertext modulus and the BFV plaintext modulus.

To describe the hypercube rotations more clearly, we partition a vector $\mathbf{m} \in \mathbb{Z}_q^N$ into two halves, and then further divide each half into $r/2$ consecutive blocks of length m . Accordingly, we index the entries by two subscripts: the first subscript records the position inside each block, ranging from 0 to $m-1$, and the second subscript records the block index, ranging from 0 to $r/2-1$. With this notation, we write $\mathbf{m}$ in the following two-dimensional hypercube form:

$$\mathbf{m} = (\begin{matrix} a_{0,0} & a_{1,0} & \cdots & a_{m-1,0} & \cdots & a_{0,r/2-1} & a_{1,r/2-1} & \cdots & a_{m-1,r/2-1} \\ b_{0,0} & b_{1,0} & \cdots & b_{m-1,0} & \cdots & b_{0,r/2-1} & b_{1,r/2-1} & \cdots & b_{m-1,r/2-1} \end{matrix}). \quad (7)$$

We represent a vector $\mathbf{m} \in \mathbb{Z}_q^N$ as an $m \times r$ matrix. More precisely, define the bijections

$$\text{mat} : \mathbb{Z}_q^N \rightarrow \mathbb{Z}_q^{m \times r}, \quad \text{rvec} : \mathbb{Z}_q^{m \times r} \rightarrow \mathbb{Z}_q^N,$$

where mat arranges the entries of $\mathbf{m}$ column by column into an $m \times r$ matrix, and rvec stacks the columns of a matrix into a vector. For $0 \leq i \leq m-1$, let $\mathbf{a}_i = (a_{i,0}, a_{i,1}, \dots, a_{i, \frac{r}{2}-1})$ and $\mathbf{b}_i = (b_{i,0}, b_{i,1}, \dots, b_{i, \frac{r}{2}-1})$. Then

$$\text{mat}(\mathbf{m}) = \begin{pmatrix} a_{0,0} & \cdots & a_{0, \frac{r}{2}-1} & b_{0,0} & \cdots & b_{0, \frac{r}{2}-1} \\ a_{1,0} & \cdots & a_{1, \frac{r}{2}-1} & b_{1,0} & \cdots & b_{1, \frac{r}{2}-1} \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ a_{m-1,0} & \cdots & a_{m-1, \frac{r}{2}-1} & b_{m-1,0} & \cdots & b_{m-1, \frac{r}{2}-1} \end{pmatrix} = \begin{pmatrix} \mathbf{a}_0 & \mathbf{b}_0 \\ \mathbf{a}_1 & \mathbf{b}_1 \\ \vdots & \vdots \\ \mathbf{a}_{m-1} & \mathbf{b}_{m-1} \end{pmatrix},$$

where each row is viewed as the concatenation of two vectors of length $r/2$.

For a vector $\mathbf{a} = (a_0, \dots, a_{\frac{r}{2}-1}) \in \mathbb{Z}_q^{r/2}$, define

$$\sigma_j(\mathbf{a}) = (a_j, a_{j+1}, \dots, a_{\frac{r}{2}-1}, a_0, a_1, \dots, a_{j-1}),$$

where the indices are taken modulo $r/2$. For $M = \text{mat}(\mathbf{m})$, define

$$\sigma_j(M) = \begin{pmatrix} \sigma_j(\mathbf{a}_0) & \sigma_j(\mathbf{b}_0) \\ \sigma_j(\mathbf{a}_1) & \sigma_j(\mathbf{b}_1) \\ \vdots & \vdots \\ \sigma_j(\mathbf{a}_{m-1}) & \sigma_j(\mathbf{b}_{m-1}) \end{pmatrix}, \quad \tau(M) = \begin{pmatrix} \mathbf{b}_0 & \mathbf{a}_0 \\ \mathbf{b}_1 & \mathbf{a}_1 \\ \vdots & \vdots \\ \mathbf{b}_{m-1} & \mathbf{a}_{m-1} \end{pmatrix}.$$

Then, for each $j = 0, 1, \dots, \frac{r}{2} - 1$, we have

$$\text{mat}(\rho_{jm}(\mathbf{m})) = \sigma_j(\text{mat}(\mathbf{m})), \quad \text{mat}(\tau(\mathbf{m})) = \tau(\text{mat}(\mathbf{m})).$$

Hence, $\mathbf{m}$ can be viewed as packing $2m$ vectors of length $r/2$, and the rotations ρ_{jm} simultaneously cyclically shift all these vectors by j positions. As a result, one can perform linear transformations on them in a batched manner.

To describe sparsely packed vectors more conveniently, we recall the notion of the Kronecker product. This notation provides a concise way to represent the repetition of a vector multiple times, and will be used throughout the paper.

Let $\mathbf{x}$ be a plaintext sparsely packing $(x_0, \dots, x_{m-1}) \in \mathbb{Z}_q^m$ with repetition multiplicity r , namely,

$$\mathbf{x} = \underbrace{(x_0, \dots, x_{m-1}, x_0, \dots, x_{m-1}, \dots, x_0, \dots, x_{m-1})}_{r \text{ times}} = \mathbf{1}_r \otimes (x_0, x_1, \dots, x_{m-1}). \quad (8)$$

Then

$$\text{mat}(\mathbf{x}) = \begin{pmatrix} x_0 & \cdots & x_0 & x_0 & \cdots & x_0 \\ x_1 & \cdots & x_1 & x_1 & \cdots & x_1 \\ \vdots & \ddots & \vdots & \vdots & \ddots & \vdots \\ x_{m-1} & \cdots & x_{m-1} & x_{m-1} & \cdots & x_{m-1} \end{pmatrix}, \quad (9)$$

where each row consists of r copies of the same entry. Therefore, $\mathbf{x}$ may be viewed as packing m row vectors, each of which has the form $(x_i, \dots, x_i) \in \mathbb{Z}_q^r$.

Recall the notion of multiplicative depth used in this paper. The depth of a ciphertext is denoted by $\text{Depth}(\text{ct})$. We assign depth 0 to every input ciphertext, and also set $\text{Depth}(\text{pt}) = 0$ for plaintexts. Only multiplications increase the multiplicative depth, while additions and rotations do not.

Recall also that, in the BGV scheme, noise growth is controlled by modulus switching along a modulus chain. Accordingly, the initial ciphertext modulus is chosen as a product of several large moduli, and each multiplication consumes one level of this chain. Therefore, if an algorithm has multiplicative depth ℓ , then the BGV ciphertext modulus must contain at least $\ell + 1$ moduli.

3 Basic algorithms

3.1 Batched linear transformation

Under the hypercube representation in Section 2.4, diagonal HLT naturally extends to the following two batched settings.

- Algorithm 1 simultaneously applies diagonal HLT to the $2m$ subvectors of length $r/2$;
- Algorithm 2 applies diagonal HLT to the m subvectors of length r by reducing this case to Algorithm 1.

As discussed in Section 2.4, a vector $\mathbf{m} \in \mathbb{Z}_q^N$ may be viewed as packing $2m$ vectors of length $r/2$, and each of these vectors admits internal cyclic rotations. Therefore, the PEGASUS homomorphic linear transformation recalled in Section 2.3 naturally extends to the batched setting, where one simultaneously applies matrix multiplication to these $2m$ vectors of length $r/2$. The resulting procedure is described in Algorithm 1.

Algorithm 1 Batched Linear Transformation over Ciphertexts with $2m$ blocks

Input: A ciphertext ct encrypting $\mathbf{m} \in \mathbb{Z}_q^N$ described in (7), with $N = rm$, and matrices $\{A_i, B_i\}_{i=0}^{m-1} \subset \mathbb{Z}_q^{\ell \times r/2}$, where $\ell \leq r/2$ and ℓ, r, m are powers of two.
Output: A ciphertext encrypting $\mathbf{y} \in \mathbb{Z}_q^N$ such that

$$\text{mat}(\mathbf{y}) = \begin{pmatrix} \mathbf{1}_s \otimes \mathbf{a}_0 A_0^T & \mathbf{1}_s \otimes \mathbf{b}_0 B_0^T \\ \mathbf{1}_s \otimes \mathbf{a}_1 A_1^T & \mathbf{1}_s \otimes \mathbf{b}_1 B_1^T \\ \vdots & \vdots \\ \mathbf{1}_s \otimes \mathbf{a}_{m-1} A_{m-1}^T & \mathbf{1}_s \otimes \mathbf{b}_{m-1} B_{m-1}^T \end{pmatrix},$$

where $s = r/(2\ell)$.

Initial: Let $\gamma \leftarrow \log s$. For each $j = 0, \dots, \ell - 1$, let $\mathbf{u}_{ij}$ and $\mathbf{v}_{ij}$ be the j -th diagonals of A_i and B_i described in Section 2.3, respectively, and define

$$\mathbf{u}_j \leftarrow \text{rvec} \begin{pmatrix} \mathbf{u}_{0,j} & \mathbf{v}_{0,j} \\ \mathbf{u}_{1,j} & \mathbf{v}_{1,j} \\ \vdots & \vdots \\ \mathbf{u}_{m-1,j} & \mathbf{v}_{m-1,j} \end{pmatrix}.$$

```

1:  $\text{ct}^{(1)} \leftarrow \text{CMult}(\text{ct}, \mathbf{u}_0)$ ;
2: for  $j = 1, \dots, \ell - 1$  do
3:    $\text{ct}^{(1)} \leftarrow \text{Add}(\text{ct}^{(1)}, \text{CMult}(\text{ColumnRotation}(\text{ct}, jm), \mathbf{u}_j))$ ;
4: end for
5:  $\text{ct}^{(2)} \leftarrow \text{ct}^{(1)}$ 
6: for  $i = 0, \dots, \gamma - 1$  do
7:    $\text{ct}^{(2)} \leftarrow \text{Add}(\text{ct}^{(2)}, \text{ColumnRotation}(\text{ct}^{(2)}, 2^i \ell m))$ ;
8: end for
9: return  $\text{ct}^{(2)}$ .

```

Algorithm 1 handles batched linear transformations on $2m$ blocks of width $r/2$. We use it as a subroutine to realize batched linear transformations on m blocks of width r . Let

$$\mathbf{c}_i = (a_{i,0}, a_{i,1}, \dots, a_{i,r/2-1}, b_{i,0}, b_{i,1}, \dots, b_{i,r/2-1}) = (\mathbf{a}_i, \mathbf{b}_i) \in \mathbb{Z}_q^r,$$

for $i = 0, 1, \dots, m - 1$. Given a ciphertext ct encrypting $\mathbf{m} \in \mathbb{Z}^N$ as in (7), and matrices $U_0, \dots, U_{m-1} \in \mathbb{Z}_q^{\ell \times r}$, our goal is to compute a ciphertext encrypting

$\mathbf{y} \in \mathbb{Z}_q^N$ such that

$$\text{mat}(\mathbf{y}) = \begin{pmatrix} \mathbf{1}_{r/\ell} \otimes \mathbf{c}_0 U_0^T \\ \mathbf{1}_{r/\ell} \otimes \mathbf{c}_1 U_1^T \\ \vdots \\ \mathbf{1}_{r/\ell} \otimes \mathbf{c}_{m-1} U_{m-1}^T \end{pmatrix}. \quad (10)$$

Algorithm 2 reduces this task to Algorithm 1 by decomposing each U_i according to the relation between ℓ and r . If $\ell < r$, write

$$U_i = (A_i, B_i), \quad A_i, B_i \in \mathbb{Z}_q^{\ell \times r/2}.$$

A single call to Algorithm 1 with input ct and $\{A_i, B_i\}_{i=0}^{m-1}$ computes the two partial products $\mathbf{1}_{r/(2\ell)} \otimes \mathbf{a}_i A_i^T$, and $\mathbf{1}_{r/(2\ell)} \otimes \mathbf{b}_i B_i^T$, for all i . Since

$$\mathbf{1}_{r/(2\ell)} \otimes \mathbf{c}_i U_i^T = \mathbf{1}_{r/(2\ell)} \otimes \mathbf{a}_i A_i^T + \mathbf{1}_{r/(2\ell)} \otimes \mathbf{b}_i B_i^T,$$

the desired ciphertext is obtained by applying the RowRotation and then adding. When $\ell = r$, write

$$U_i = \begin{pmatrix} A_i & B_i \\ C_i & D_i \end{pmatrix}, \quad A_i, B_i, C_i, D_i \in \mathbb{Z}_q^{r/2 \times r/2}, \quad i = 0, \dots, m-1.$$

Then $\mathbf{c}_i U_i^T = (\mathbf{a}_i A_i^T + \mathbf{b}_i B_i^T, \mathbf{a}_i C_i^T + \mathbf{b}_i D_i^T)$. Accordingly, we apply Algorithm 1 twice: first to ct with $\{A_i, D_i\}_{i=0}^{m-1}$, and then to $\text{RowRotation}(\text{ct}, 1)$ with $\{B_i, C_i\}_{i=0}^{m-1}$. Therefore, $\text{Add}(\text{ct}^{(1)}, \text{ct}^{(2)})$ encrypts the desired vector. In this way, Algorithm 2 reduces the case of m blocks of width r to the case of $2m$ blocks of width $r/2$ handled by Algorithm 1.

Complexity of Algorithm 2. If $\ell < r$, then Algorithm 2 requires ℓ plaintext-ciphertext multiplications and $\ell + \log(r/(2\ell))$ homomorphic rotations. Moreover, if $\ell = bg$, then by the baby-step giant-step strategy, the number of homomorphic rotations can be reduced to $b + g - 1 + \log(r/(2\ell))$.

If $\ell = r$, then Algorithm 2 requires r plaintext-ciphertext multiplications and $r - 1$ homomorphic rotations. Moreover, if $r/2 = bg$, then by the baby-step giant-step strategy, the number of homomorphic rotations can be reduced to $2b + 2g - 3$.

Moreover, in both cases, the multiplicative depth is 1.

3.2 Batched generating powers

Let ct be a ciphertext encrypting $\mathbf{x} = \mathbf{1}_r \otimes (x_0, x_1, \dots, x_{m-1})$. As discussed in (9), the vector $\mathbf{x}$ may be viewed as packing m row vectors, each of the form $(x_i, \dots, x_i) \in \mathbb{Z}_q^r$. Following the technique reviewed in Section 1.3, we now explain how to generate powers in a batched manner. Let $s \leq r$, and assume

Algorithm 2 BatchLT(ct, $\{U_0, \dots, U_{m-1}\}$)

Input: A ciphertext $\mathbf{ct}$ encrypting $\mathbf{m} \in \mathbb{Z}_q^N$ as in (7), with $N = rm$, and matrices $U_0, \dots, U_{m-1} \in \mathbb{Z}_q^{\ell \times r}$, where $\ell \leq r$, and ℓ, r, m are powers of two.

Output: A ciphertext encrypting $\mathbf{y} \in \mathbb{Z}_q^N$ such that $\text{mat}(\mathbf{y})$ is described as (10)

1: **if** $\ell < r$ **then**

2: Write $U_i = (A_i, B_i)$ with $A_i, B_i \in \mathbb{Z}_q^{\ell \times r/2}$ for $i = 0, \dots, m-1$.

3: Compute $\mathbf{ct}^{(1)}$ by Algorithm 1 with input $\mathbf{ct}$ and $\{A_i, B_i\}_{i=0}^{m-1}$.

4: **return** Add(ct⁽¹⁾, RowRotation(ct⁽¹⁾, 1)).

5: else

6: Write

$$U_i = \begin{pmatrix} A_i & B_i \\ C_i & D_i \end{pmatrix}$$

with $A_i, B_i, C_i, D_i \in \mathbb{Z}_q^{r/2 \times r/2}$ for $i = 0, \dots, m-1$.

7: Set $ct' \leftarrow \text{RowRotation}(ct)$.

8: Compute $\text{ct}^{(1)}$ by Algorithm 1 with input ct and $\{A_i, D_i\}_{i=0}^{m-1}$.

9: Compute $\text{ct}^{(2)}$ by Algorithm 1 with input ct' and $\{B_i, C_i\}_{i=0}^{m-1}$.

```

10:   return Add(ct(1), ct(2)).

```

```

11: end if

```

that s is a power of two. Our goal is to compute a ciphertext $\mathbf{ct}_{\text{out}}$ encrypting a vector $\mathbf{y} \in \mathbb{Z}_q^N$ with

$$\text{mat}(\mathbf{y}) = \begin{pmatrix} \mathbf{1}_{r/s} \otimes (1, x_0, x_0^2, \dots, x_0^{s-1}) \\ \mathbf{1}_{r/s} \otimes (1, x_1, x_1^2, \dots, x_1^{s-1}) \\ \vdots \\ \mathbf{1}_{r/s} \otimes (1, x_{m-1}, x_{m-1}^2, \dots, x_{m-1}^{s-1}) \end{pmatrix}. \quad (11)$$

The concrete procedure is given in Algorithm 3.

Complexity of Algorithm 3. The plaintext-ciphertext multiplications occur in Line 6 and Line 9, the number of which is 1 and $\log(s) - 1$, respectively, so the total number of plaintext-ciphertext multiplications is $\log(s)$. The ciphertext-ciphertext multiplications occur in Line 4 and Line 10, the number of which are $\log(s) - 1$ and $\log(s) - 1$, respectively, so the total number of ciphertext-ciphertext multiplications is $2\log(s) - 2$.

From Line 4, we have $\text{Depth}(\text{ct}_i) = i$, for $i = 1, \dots, \ell - 1$. Furthermore, $\text{Depth}(\text{ct}'_i) = i + 1$ for $i = 1, \dots, \ell - 1$. Therefore, in the i -th loop in Line 10, $\text{Depth}(\text{ct}'') = i + 2$ for $i = 1, \dots, \ell - 1$. Then the ciphertext ct'' returned in Lines 13 and 15 has depth $\ell + 1$.

In the following, we illustrate the construction in the case $N = 16$, $s = 4$, and $r = 8$. For simplicity, we expand it at the plaintext level.

Example 1. Let $N = 16$, $s = 4$, and $m = 2$. Then $r = 8$ and $t = 2$. Let

[illegible]

Algorithm 3 EvalPower(ct, s, b)

Input: A ciphertext ct encrypting $\mathbf{1}_r \otimes (x_0, \dots, x_{m-1})$, an integer $s \leq r$, where m, r, s are powers of two, and an integer $b \in \{0, 1\}$. (The auxiliary output for $b = 1$ is used later to generate giant step powers in the case $r < d + 1 \leq r^2$ in Section 4.2).
Output: A ciphertext encrypting $\mathbf{y} \in \mathbb{Z}_q^N$ such that $\text{mat}(\mathbf{y})$ is given in (11). In addition, if $b = 1$, the algorithm also outputs a ciphertext encrypting $\mathbf{1}_r \otimes (x_0^{s/2}, \dots, x_{m-1}^{s/2})$.

```

1: Initialization: Set  $s = 2^\ell$  and  $t = r/s$ . Let  $C \in \mathbb{Z}_q^{\ell \times s}$  be the matrix whose columns
   are the binary expansions of  $0, 1, \dots, s-1$ , and let  $D = \mathbf{1}_{\ell \times s} - C$ . Denote the  $i$ -th
   rows of  $C$  and  $D$  by  $\mathbf{c}_i$  and  $\mathbf{d}_i$ , respectively. Let  $C_i^{\text{ext}} \in \mathbb{Z}_q^{m \times r}$  be the matrix whose
   every row is  $\mathbf{1}_t \otimes \mathbf{c}_i$ . Let  $D_i^{\text{ext}} \in \mathbb{Z}_q^{m \times r}$  be the matrix whose every row is  $\mathbf{1}_t \otimes \mathbf{d}_i$ .
    $\mathbf{c}_i^{\text{ext}} \leftarrow \text{rvec}(C_i^{\text{ext}})$ .  $\mathbf{d}_i^{\text{ext}} \leftarrow \text{rvec}(D_i^{\text{ext}})$ .
2:  $\text{ct}_0 \leftarrow \text{ct}$ .
3: for  $i = 1, \dots, \ell - 1$  do
4:    $\text{ct}_i \leftarrow \text{Mult}(\text{ct}_{i-1}, \text{ct}_{i-1})$ .
5: end for
6:  $\text{ct}'_0 \leftarrow \text{Add}(\text{CMult}(\text{ct}_0, \mathbf{c}_0^{\text{ext}}), \mathbf{d}_0^{\text{ext}})$ .
7:  $\text{ct}'' \leftarrow \text{ct}'_0$ .
8: for  $i = 1, \dots, \ell - 1$  do
9:    $\text{ct}'_i \leftarrow \text{Add}(\text{CMult}(\text{ct}_i, \mathbf{c}_i^{\text{ext}}), \mathbf{d}_i^{\text{ext}})$ .
10:   $\text{ct}'' \leftarrow \text{Mult}(\text{ct}'', \text{ct}'_i)$ .
11: end for
12: if  $b = 0$  then
13:   return  $\text{ct}''$ .
14: else
15:   return  $\text{ct}'', \text{ct}_{\ell-1}$ .
16: end if

```

be a plaintext sparsely packed in $\mathbb{Z}_q^{16}$. Then

$$\text{mat}(\mathbf{x}) = \begin{pmatrix} x_0 & x_0 & x_0 & x_0 & x_0 & x_0 & x_0 & x_0 \\ x_1 & x_1 & x_1 & x_1 & x_1 & x_1 & x_1 & x_1 \end{pmatrix}.$$

Let $C = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}$, and $D = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix}$. The extended matrices associated with the two rows of C and D are

$$C_0^{\text{ext}} = \begin{pmatrix} 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 \end{pmatrix}, \quad C_1^{\text{ext}} = \begin{pmatrix} 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 \end{pmatrix},$$

$$D_0^{\text{ext}} = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \end{pmatrix}, \quad D_1^{\text{ext}} = \begin{pmatrix} 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \end{pmatrix}.$$

Let $\mathbf{c}_i^{\text{ext}} = \text{rvec}(C_i^{\text{ext}})$ and $\mathbf{d}_i^{\text{ext}} = \text{rvec}(D_i^{\text{ext}})$ for $i = 0, 1$. Set $\mathbf{x}_0 = \mathbf{x}$, $\mathbf{x}_1 = \mathbf{x}_0 \odot \mathbf{x}_0$, and define $\mathbf{y}_0 = \mathbf{x}_0 \odot \mathbf{c}_0^{\text{ext}} + \mathbf{d}_0^{\text{ext}}$ and $\mathbf{y}_1 = \mathbf{x}_1 \odot \mathbf{c}_1^{\text{ext}} + \mathbf{d}_1^{\text{ext}}$. Then

$$\text{mat}(\mathbf{y}_0) = \begin{pmatrix} 1 & x_0 & 1 & x_0 & 1 & x_0 & 1 & x_0 \\ 1 & x_1 & 1 & x_1 & 1 & x_1 & 1 & x_1 \end{pmatrix}, \quad \text{mat}(\mathbf{y}_1) = \begin{pmatrix} 1 & 1 & x_0^2 & x_0^2 & 1 & 1 & x_0^2 & x_0^2 \\ 1 & 1 & x_1^2 & x_1^2 & 1 & 1 & x_1^2 & x_1^2 \end{pmatrix}.$$

Let $\mathbf{y} = \mathbf{y}_0 \odot \mathbf{y}_1$. Then we obtain

$$\text{mat}(\mathbf{y}) = \begin{pmatrix} 1 & x_0 & x_0^2 & x_0^3 & 1 & x_0 & x_0^2 & x_0^3 \\ 1 & x_1 & x_1^2 & x_1^3 & 1 & x_1 & x_1^2 & x_1^3 \end{pmatrix},$$

and the corresponding vector is

$$\mathbf{y} = (1, 1, x_0, x_1, x_0^2, x_1^2, x_0^3, x_1^3, 1, 1, x_0, x_1, x_0^2, x_1^2, x_0^3, x_1^3).$$

4 Algorithms for homomorphic polynomial evaluation

In this section, we will present an algorithm for Problem 1 when $d + 1 \leq r^2$.

4.1 The Case $d + 1 \leq r$

As shown in Section 3.2 and Algorithm 3, when $d + 1 \leq r$, one can homomorphically generate a ciphertext encrypting all powers $1, x_i, x_i^2, \dots, x_i^d$ for each x_i . Therefore, to evaluate the polynomials $(f_0, f_1, \dots, f_{m-1})$, it remains to multiply these powers by the corresponding coefficients of $(f_0, \dots, f_{m-1})$ and then combine the resulting monomials by the standard rotate-and-sum procedure. This yields a ciphertext encrypting the desired polynomial evaluations. The detailed procedure is given in Algorithm 4.

Complexity of Algorithm 4. The call to `EvalPower(ct, d + 1, 0)` in Line 1 requires $2 \log(d + 1) - 2$ ciphertext-ciphertext multiplications, $\log(d + 1)$ plaintext-ciphertext multiplications, and multiplicative depth $\log(d + 1) + 1$. Line 2 contributes one additional plaintext-ciphertext multiplication, so the ciphertext ct'_0 has depth $\log(d + 1) + 2$. The remaining steps involve only additions and rotations, and therefore do not increase the multiplicative depth. Hence, Algorithm 4 uses $2 \log(d + 1) - 2$ ciphertext-ciphertext multiplications, $\log(d + 1) + 1$ plaintext-ciphertext multiplications, and $\log(d + 1)$ homomorphic rotations, and its multiplicative depth is $\log(d + 1) + 2$.

Remark 1. As in Section 1.3, one may incorporate the coefficients of the polynomials f_i directly into the matrices C used in Algorithm 3. In this way, the plaintext-ciphertext multiplication in Line 2 can be merged into the power-generation step. Consequently, the multiplicative depth of Algorithm 4 becomes $\log(d + 1) + 1$, and the number of plaintext-ciphertext multiplications is reduced by one.

Algorithm 4 Homomorphic polynomial evaluation for sparsely packed ciphertexts when $d + 1 \leq r$

Input: A ciphertext ct encrypting $\mathbf{1}_r \otimes (x_0, \dots, x_{m-1})$, and polynomials $f_0, \dots, f_{m-1} \in \mathbb{Z}_q[x]$ of degree at most $d < r$ with $f_i(x) = \sum_{j=0}^d a_{ij}x^j$;

Output: A ciphertext encrypting $\mathbf{1}_r \otimes (f_0(x_0), f_1(x_1), \dots, f_{m-1}(x_{m-1}))$ with repetition multiplicity r .

Initial: Let $d + 1 = 2^\ell$ and $\mathbf{a}_i = (a_{i0}, a_{i1}, \dots, a_{id})$ for $i = 0, \dots, m - 1$. Let $\mathbf{f} \in \mathbb{Z}_q^N$ with

$$\text{mat}(\mathbf{f}) = \begin{pmatrix} \mathbf{1}_{r/(d+1)} \otimes (a_{0,0}, a_{0,1}, a_{0,2}, \dots, a_{0,d}) \\ \mathbf{1}_{r/(d+1)} \otimes (a_{1,0}, a_{1,1}, a_{1,2}, \dots, a_{1,d}) \\ \vdots \\ \mathbf{1}_{r/(d+1)} \otimes (a_{m-1,0}, a_{m-1,1}, a_{m-1,2}, \dots, a_{m-1,d}) \end{pmatrix}.$$

```

1:  $\text{ct}' \leftarrow \text{EvalPower}(\text{ct}, d + 1, 0)$ ;
2:  $\text{ct}'_0 \leftarrow \text{CMult}(\text{ct}', \mathbf{f})$ ;
3: for  $i = 1, \dots, \ell - 1$  do
4:    $\text{ct}'_i \leftarrow \text{Add}(\text{ct}'_{i-1}, \text{ColumnRotation}(\text{ct}'_{i-1}, m2^{i-1}))$ ;
5: end for
6: if  $d + 1 = r$  then
7:    $\text{ct}'_\ell \leftarrow \text{Add}(\text{ct}'_{\ell-1}, \text{RowRotation}(\text{ct}'_{\ell-1}, 1))$ ;
8: else
9:    $\text{ct}'_\ell \leftarrow \text{Add}(\text{ct}'_{\ell-1}, \text{ColumnRotation}(\text{ct}'_{\ell-1}, m2^{\ell-1}))$ ;
10: end if
11: return  $\text{ct}'_\ell$ .
```

We now illustrate the procedure of Algorithm 4 through a concrete example.

Example 2. Let $N = 16$, $d = 3$, and $m = 2$. Then $r = 8$. Consider

$$\mathbf{x} = (x_0, x_1, x_0, x_1, x_0, x_1, x_0, x_1, x_0, x_1, x_0, x_1, x_0, x_1, x_0, x_1) = \mathbf{1}_8 \otimes (x_0, x_1).$$

By Algorithm 3, we obtain a ciphertext ct' encrypting a vector $\mathbf{y}$ such that

$$\text{mat}(\mathbf{y}) = \begin{pmatrix} 1 & x_0 & x_0^2 & x_0^3 & 1 & x_0 & x_0^2 & x_0^3 \\ 1 & x_1 & x_1^2 & x_1^3 & 1 & x_1 & x_1^2 & x_1^3 \end{pmatrix}.$$

Let $\mathbf{f} \in \mathbb{Z}_q^{16}$ be such that

$$\text{mat}(\mathbf{f}) = \begin{pmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} & a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} & a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \end{pmatrix}.$$

After computing $\text{ct}'_0 \leftarrow \text{CMult}(\text{ct}', \mathbf{f})$, we obtain a ciphertext ct'_0 encrypting a vector $\mathbf{y}'$ with

$$\text{mat}(\mathbf{y}') = \begin{pmatrix} a_{0,0} & a_{0,1}x_0 & a_{0,2}x_0^2 & a_{0,3}x_0^3 & a_{0,0} & a_{0,1}x_0 & a_{0,2}x_0^2 & a_{0,3}x_0^3 \\ a_{1,0} & a_{1,1}x_1 & a_{1,2}x_1^2 & a_{1,3}x_1^3 & a_{1,0} & a_{1,1}x_1 & a_{1,2}x_1^2 & a_{1,3}x_1^3 \end{pmatrix}.$$

Now set

$$\text{ct}'_1 \leftarrow \text{Add}(\text{ct}'_0, \text{ColumnRotation}(\text{ct}'_0, 2)), \quad \text{ct}'_2 \leftarrow \text{Add}(\text{ct}'_1, \text{ColumnRotation}(\text{ct}'_1, 4)).$$

In this way, the monomials $a_{0,0}, a_{0,1}x_0, a_{0,2}x_0^2, a_{0,3}x_0^3$ and $a_{1,0}, a_{1,1}x_1, a_{1,2}x_1^2, a_{1,3}x_1^3$ are summed separately. As a result, ct'_2 encrypts a vector $\mathbf{h}$ such that

$$\text{mat}(\mathbf{h}) = \begin{pmatrix} f_0(x_0) & f_0(x_0) & f_0(x_0) & f_0(x_0) & f_0(x_0) & f_0(x_0) & f_0(x_0) & f_0(x_0) \\ f_1(x_1) & f_1(x_1) & f_1(x_1) & f_1(x_1) & f_1(x_1) & f_1(x_1) & f_1(x_1) & f_1(x_1) \end{pmatrix},$$

and hence $\mathbf{h} = \mathbf{1}_8 \otimes (f_0(x_0), f_1(x_1))$.

4.2 The case $r < d + 1 \leq r^2$

Suppose $r < d + 1 \leq r^2$ and let $t = (d + 1)/r$. We assume that $d + 1$, r , and t are all powers of two. For each $k = 0, \dots, m - 1$, the Paterson–Stockmeyer method [51] decomposes $f_k(x)$ as

$$g_{k,i}(x) = a_{k,ir} + a_{k,ir+1}x + \dots + a_{k,ir+r-1}x^{r-1}, \quad i = 0, \dots, t - 1.$$

Then $f_k(x)$ can be written as

$$f_k(x) = g_{k,0}(x) + g_{k,1}(x)x^r + \dots + g_{k,t-1}(x)x^{(t-1)r}.$$

Based on Algorithm 3, our algorithm for the case $r < d + 1 \leq r^2$ consists of three steps.

Step 1. Generation of the power vectors. Using Algorithm 3, we first compute two ciphertexts ct_1 and ct_2 , encrypting vectors $\mathbf{y}_1$ and $\mathbf{y}_2$, respectively, such that

$$\text{mat}(\mathbf{y}_1) = \begin{pmatrix} 1 & x_0 & x_0^2 & \dots & x_0^{r-1} \\ 1 & x_1 & x_1^2 & \dots & x_1^{r-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{m-1} & x_{m-1}^2 & \dots & x_{m-1}^{r-1} \end{pmatrix}, \quad \text{mat}(\mathbf{y}_2) = \begin{pmatrix} \mathbf{1}_{r/t} \otimes (1, x_0^r, x_0^{2r}, \dots, x_0^{(t-1)r}) \\ \mathbf{1}_{r/t} \otimes (1, x_1^r, x_1^{2r}, \dots, x_1^{(t-1)r}) \\ \vdots \\ \mathbf{1}_{r/t} \otimes (1, x_{m-1}^r, x_{m-1}^{2r}, \dots, x_{m-1}^{(t-1)r}) \end{pmatrix}.$$

Step 2. Evaluation of the block polynomials. For each $k = 0, \dots, m - 1$, define the coefficient matrix

$$G_k = \begin{pmatrix} a_{k,0} & a_{k,1} & \dots & a_{k,r-1} \\ a_{k,r} & a_{k,r+1} & \dots & a_{k,2r-1} \\ \vdots & \vdots & & \vdots \\ a_{k,(t-1)r} & a_{k,(t-1)r+1} & \dots & a_{k,tr-1} \end{pmatrix}. \quad (12)$$

Then, by applying the batch linear transformation algorithm, Algorithm 2 to ct_1 , we obtain a ciphertext ct_3 encrypting a vector $\mathbf{z}$ such that

$$\text{mat}(\mathbf{z}) = \begin{pmatrix} \mathbf{1}_{r/t} \otimes (g_{0,0}(x_0), g_{0,1}(x_0), \dots, g_{0,t-1}(x_0)) \\ \mathbf{1}_{r/t} \otimes (g_{1,0}(x_1), g_{1,1}(x_1), \dots, g_{1,t-1}(x_1)) \\ \vdots \\ \mathbf{1}_{r/t} \otimes (g_{m-1,0}(x_{m-1}), g_{m-1,1}(x_{m-1}), \dots, g_{m-1,t-1}(x_{m-1})) \end{pmatrix}.$$

Step 3. Recombination. Next, we multiply ct_3 and ct_2 . Finally, applying the rotate-and-sum procedure within each row yields a ciphertext encrypting

$$\mathbf{1}_r \otimes (f_0(x_0), f_1(x_1), \dots, f_{m-1}(x_{m-1})).$$

We summarize the above construction for the case $r < d + 1 \leq r^2$ in Algorithm 5.

Algorithm 5 Homomorphic Polynomial evaluation for sparsely packed ciphertexts when $r < d + 1 \leq r^2$

Input: A ciphertext ct encrypting $\mathbf{x} = \mathbf{1}_r \otimes (x_0, \dots, x_{m-1})$, polynomials $f_0, \dots, f_{m-1} \in \mathbb{Z}_q[x]$ of degree at most d , where $f_i(x) = \sum_{j=0}^d a_{ij}x^j$ and $r < d + 1 \leq r^2$;

Output: A ciphertext encrypting $\mathbf{1}_r \otimes (f_0(x_0), f_1(x_1), \dots, f_{m-1}(x_{m-1}))$.

Initial: Let $s = (d + 1)/r$ and $\ell = \log(s)$.

```

1:  $\text{ct}_1, \text{ct}' \leftarrow \text{EvalPower}(\text{ct}, r, 1)$ ;
2:  $\text{ct}' \leftarrow \text{Mult}(\text{ct}', \text{ct}')$ ;
3:  $\text{ct}_2 \leftarrow \text{EvalPower}(\text{ct}', s, 0)$ ;
4:  $\text{ct}_3 \leftarrow \text{BatchLT}(\text{ct}_1, \{G_0, G_1, \dots, G_{m-1}\})$ ;  $G_k$  are defined in (12);
5:  $\text{ct}^{(0)} \leftarrow \text{Mult}(\text{ct}_2, \text{ct}_3)$ ;
6: for  $i = 1, \dots, \ell - 1$  do
7:    $\text{ct}^{(i)} \leftarrow \text{Add}(\text{ct}^{(i-1)}, \text{ColumnRotation}(\text{ct}^{(i-1)}, m2^{i-1}))$ ;
8: end for
9: if  $d + 1 = r^2$  then
10:    $\text{ct}^{(\ell)} \leftarrow \text{Add}(\text{ct}^{(\ell-1)}, \text{RowRotation}(\text{ct}^{(\ell-1)}, 1))$ ;
11: else
12:    $\text{ct}^{(\ell)} \leftarrow \text{Add}(\text{ct}^{(\ell-1)}, \text{ColumnRotation}(\text{ct}^{(\ell-1)}, m2^{\ell-1}))$ ;
13: end if
14: return  $\text{ct}^{(\ell)}$ ;
```

Complexity of Algorithm 5. Recall $t = (d + 1)/r = m(d + 1)/N$. The two calls to EvalPower in Lines 1 and 2 require in total $2 \log(d + 1) - 4$ ciphertext-ciphertext multiplications and $\log(d + 1)$ plaintext-ciphertext multiplications. The number of plaintext-ciphertext multiplications in Line 4 is t . The multiplication in Line 2 and Line 5 contributes two more ciphertext-ciphertext multiplications, while the remaining operations are only additions and rotations. Therefore, the total number of ciphertext-ciphertext multiplications is $2 \log(d + 1) - 2$, the total number of plaintext-ciphertext multiplications is $t + \log(d + 1)$, and the multiplicative depth is $\log(d + 1) + 2$.

For the homomorphic rotations, if $d + 1 < r^2$, then the algorithm uses $t + \log r - 1$ rotations. Moreover, if $t = bg$, then by applying the baby-step giant-step strategy to Algorithm 2 in Line 4, this number can be reduced to $b + g - 2 + \log r$. If $d + 1 = r^2$, then the algorithm uses $r + \log r - 1$ rotations. Moreover, if $r/2 = bg$, then by applying the baby-step giant-step strategy to Algorithm 2 in Line 4, this number can be reduced to $2b + 2g + \log r - 3$.

5 Applications in Functional Bootstrapping

In this section, we apply the homomorphic polynomial evaluation algorithms developed in the previous section to the batched functional bootstrapping framework of Liu and Wang [44]. Recall that functional bootstrapping refers to a procedure that, given an LWE ciphertext encrypting a message $m \in \mathbb{Z}_p$, outputs a refreshed LWE ciphertext encrypting $f(m)$ for a target function $f : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$, while keeping the output noise within the correct decryption range. In particular, the algorithm of [44] takes as input a batch of LWE ciphertexts encrypting messages in $\mathbb{Z}_p$, and returns refreshed LWE ciphertexts encrypting the values of f on the same messages.

At a high level, the construction of [44] consists of two main stages. In the first stage, it homomorphically computes $b - \langle \mathbf{a}, \mathbf{s} \rangle$ in the BFV scheme and then evaluates a polynomial f_{poly} representing the target lookup table. In the second stage, it converts the resulting BFV ciphertext back to LWE ciphertexts by applying SLOTTOCOEFF, SAMPLEEXTRACT, key switching, and modulus switching.

We make a slight modification to this framework. We consider a batch size m satisfying $m \leq \lceil N/\sqrt{q} \rceil$, and assume that m is a power of two. Under this parameter choice, the first stage produces a sparsely packed BFV ciphertext rather than a fully packed one. We then replace the polynomial-evaluation step in [44] by our homomorphic polynomial evaluation algorithm for sparsely packed ciphertexts. For the SLOTTOCOEFF step, we use a sparse version analogous to that of [12].

For clarity, we divide the modified construction into the following three steps.

Step 1: Homomorphic decryption and sparse packing. Let $r = N/m$. The first step is a sparse-packed version of the homomorphic decryption stage in the batched functional bootstrapping framework of [44]. It takes m LWE ciphertexts $\text{ct}_i = (\mathbf{a}_i, b_i) \in \mathbb{Z}_q^{n+1}$ with $b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + \alpha\mu_i + e_i \pmod{q}$, for $i = 0, \dots, m-1$, and outputs a BFV ciphertext encrypting

$$\mathbf{1}_r \otimes (\alpha\mu_0 + e_0, \alpha\mu_1 + e_1, \dots, \alpha\mu_{m-1} + e_{m-1}). \quad (13)$$

If $r \geq n$, let ct be a BFV ciphertext encrypting $\mathbf{s}^{(\text{ext})}$, and let $\mathbf{a} \in \mathbb{Z}_q^N$, where

$$\text{mat}(\mathbf{s}^{(\text{ext})}) = \begin{pmatrix} \mathbf{1}_{r/n} \otimes \mathbf{s} \\ \vdots \\ \mathbf{1}_{r/n} \otimes \mathbf{s} \end{pmatrix}, \quad \text{mat}(\mathbf{a}) = \begin{pmatrix} \mathbf{1}_{r/n} \otimes \mathbf{a}_0 \\ \vdots \\ \mathbf{1}_{r/n} \otimes \mathbf{a}_{m-1} \end{pmatrix}.$$

Then $\text{CMult}(\text{ct}, -\mathbf{a})$ is a BFV ciphertext encrypting the slotwise products $-a_{ij}s_j$. After applying rotations and summing, and then adding $\mathbf{b}$, where the i -th row of $\text{mat}(\mathbf{b})$ is $(b_i, b_i, \dots, b_i) \in \mathbb{Z}_q^r$, we obtain a BFV ciphertext encrypting (13).

If $n > r$, let $t = n/r = mn/N$. Write

$$\mathbf{s} = (\mathbf{s}^{(0)}, \mathbf{s}^{(1)}, \dots, \mathbf{s}^{(t-1)}), \quad \mathbf{a}_i = (\mathbf{a}_i^{(0)}, \mathbf{a}_i^{(1)}, \dots, \mathbf{a}_i^{(t-1)}),$$

where $\mathbf{s}^{(k)}, \mathbf{a}_i^{(k)} \in \mathbb{Z}_q^r$. For each $k = 0, \dots, t-1$, let ct_k be a BFV ciphertext encrypting $\mathbf{s}_k^{(\text{ext})}$, and let $\mathbf{a}^{(k)} \in \mathbb{Z}_q^N$ satisfy

$$\text{mat}(\mathbf{s}_k^{(\text{ext})}) = \begin{pmatrix} \mathbf{s}^{(k)} \\ \vdots \\ \mathbf{s}^{(k)} \end{pmatrix}, \quad \text{mat}(\mathbf{a}^{(k)}) = \begin{pmatrix} \mathbf{a}_0^{(k)} \\ \vdots \\ \mathbf{a}_{m-1}^{(k)} \end{pmatrix}.$$

For each $k = 0, \dots, t-1$, $\text{CMult}(\text{ct}_k, -\mathbf{a}^{(k)})$ encrypts a vector whose i -th row is $-\mathbf{a}_i^{(k)} \odot \mathbf{s}^{(k)}$. Summing these ciphertexts over k and then applying a single rotate-and-sum within each row yields a BFV ciphertext encrypting

$$\mathbf{1}_r \otimes (-\langle \mathbf{a}_0, \mathbf{s} \rangle, \dots, -\langle \mathbf{a}_{m-1}, \mathbf{s} \rangle).$$

Finally, adding $\mathbf{b}$, where the i -th row of $\text{mat}(\mathbf{b})$ is $(b_i, b_i, \dots, b_i) \in \mathbb{Z}_q^r$, gives a BFV ciphertext encrypting (13).

Step 2. Homomorphic polynomial evaluation on sparse ciphertexts.

Following the construction of Liu and Wang [44], we work with BFV plaintext modulus q . Let $\alpha = \lfloor q/p \rfloor$ and $B = \lfloor (q/(2p)) \rfloor$. For each $\mu \in \mathbb{Z}_p$, define

$$I_\mu = \{ \alpha\mu + e \pmod{q} : |e| < B \} \subseteq \mathbb{Z}_q.$$

Since $\alpha = \lfloor q/p \rfloor$ and $B = \lfloor q/(2p) \rfloor$, the sets I_μ are disjoint. We then define a lookup table $\text{LUT} : \mathbb{Z}_q \rightarrow \mathbb{Z}_q$ by $\text{LUT}(x) = \alpha f(\mu)$, for all $x \in I_\mu$, and assign zero to $\text{LUT}(x)$ outside $\bigcup_{\mu \in \mathbb{Z}_p} I_\mu$. Since q is prime, every function on $\mathbb{Z}_q$ is represented by a unique polynomial of degree at most $q-1$. In particular, we define

$$f_{\text{poly}}(X) = \text{LUT}(0) - \sum_{i=1}^{q-1} X^i \sum_{a=0}^{q-1} \text{LUT}(a) a^{q-1-i}.$$

Then $f_{\text{poly}}(x) = \text{LUT}(x)$ for every $x \in \mathbb{Z}_q$. Therefore, for every $x = \alpha\mu + e \in \mathbb{Z}_q$ with $|e| < B$, we have $f_{\text{poly}}(x) = \alpha f(\mu)$. Hence, homomorphically evaluating f_{poly} on a BFV ciphertext encrypting

$$\mathbf{1}_r \otimes (\alpha\mu_0 + e_0, \alpha\mu_1 + e_1, \dots, \alpha\mu_{m-1} + e_{m-1}),$$

where $|e_i| < B$, obtains a BFV ciphertext encrypting

$$\mathbf{1}_r \otimes (\alpha f(\mu_0), \alpha f(\mu_1), \dots, \alpha f(\mu_{m-1})). \quad (14)$$

Step 3. Sparse SLOTToCOEFF and extraction back to LWE ciphertexts. Let $N = 2tm$, so that $t = r/2$. After Step 2, the BFV plaintext has the repeated-slot form

$$\mathbf{x} = \mathbf{1}_r \otimes (x_0, x_1, \dots, x_{m-1}) \in \mathbb{Z}_q^N, \quad x_i = \alpha f(\mu_i).$$

Up to a fixed public permutation of the slots, this vector is a t -sparse packing together with the row-invariance condition $\tau(\mathbf{x}) = \mathbf{x}$. In the sparse-packing framework of [12], a t -sparse plaintext is supported on the subring

$$\mathbb{Z}_q[X^t]/(X^N + 1) \cong \mathbb{Z}_q[Y]/(Y^{2m} + 1), \quad Y = X^t.$$

The additional condition $\tau(\mathbf{x}) = \mathbf{x}$ restricts this sparse subring to its τ -invariant part. Equivalently, the relevant coefficient space is spanned by plaintexts of the form

$$a_0 + \sum_{j=1}^{m-1} a_j (X^{jt} - X^{(2m-j)t}).$$

This is analogous to the conjugate-invariant encoding of [34].

We use the SLOTTOCOEFF map restricted to this m -dimensional repeated-slot subspace. The ordinary restricted SLOTTOCOEFF map and the map that sends $\mathbf{1}_r \otimes (u_0, \dots, u_{m-1})$ to

$$u_0 + \sum_{j=1}^{m-1} u_j (X^{jt} - X^{(2m-j)t})$$

differ only by a public invertible $m \times m$ linear change of basis on the sparse slots. We absorb this public change of basis, together with the usual BFV encoding normalizations, into the plaintext diagonals of the homomorphic linear transform. Hence, after applying the sparse SLOTTOCOEFF transform, the coefficient of X^{tj} is $\alpha f(\mu_j)$, for $j = 0, \dots, m-1$, while the conjugate positions $X^{(2m-j)t}$ carry the corresponding negative coefficients.

Since the restricted transform is linear and has only m independent diagonals, it can be evaluated by the standard diagonal method with m plaintext-ciphertext multiplications, and by the usual baby-step giant-step optimization with $O(\sqrt{m})$ rotations, as in the sparse SLOTTOCOEFF procedures used in CKKS bootstrapping [12]. Finally, following Liu and Wang [44, Section 4.5], we perform BFV key switching and modulus switching before extraction. We then apply SAMPLEEXTRACT only to the coefficients of X^{tj} , for $j = 0, \dots, m-1$, which yields m LWE ciphertexts encrypting $f(\mu_0), \dots, f(\mu_{m-1})$.

Summary. Combining the sparse homomorphic decryption in Step 1, the sparse polynomial evaluation in Step 2, and the sparse SLOTTOCOEFF/extraction procedure in Step 3, we obtain the following formal guarantee for the resulting functional bootstrapping construction.

Let

$$\sigma_{\text{res}} := \sqrt{\left(\frac{q}{Q'}(\sigma + \sigma_{\text{ks}})\right)^2 + \sigma_{\text{ms}}^2},$$

where σ is the standard deviation of the coefficient noise after Step 2 and the sparse SLOTTOCOEFF procedure but before the BFV key-switching step, Q' is the BFV ciphertext modulus after the BFV key-switching step, σ_{ks} is the

additional noise introduced by the BFV key switching, and σ_{ms} is the rounding noise introduced by the final LWE modulus-switching step.

Theorem 2. *Adopt the notation and standing assumptions of Section 2.4. Let $f : \mathbb{Z}_p \rightarrow \mathbb{Z}_p$ be any function, and let $m \leq N/\sqrt{d+1}$. Suppose that $\text{ct}_i = (\mathbf{a}_i, b_i) \in \mathbb{Z}_q^{n+1}$, for $i = 0, \dots, m-1$, are LWE ciphertexts encrypting $\mu_i \in \mathbb{Z}_p$ under the same LWE secret key $\mathbf{s}$, with error magnitude less than $\lfloor q/(2p) \rfloor$. Assume moreover that the BFV parameters are chosen so that every intermediate BFV ciphertext in Steps 1–3 remains decryptable. Then the construction in Section 5 outputs LWE ciphertexts*

$$\text{ct}_i^{\text{out}} = (\mathbf{a}_i^{\text{out}}, b_i^{\text{out}}) \in \mathbb{Z}_q^{n+1}, \quad i = 0, \dots, m-1,$$

encrypting $f(\mu_i)$ under the same secret key $\mathbf{s}$, each with output-noise standard deviation at most σ_{res} , for $i = 0, \dots, m-1$.

Proof (Proof sketch). We prove the theorem by correctness and noise analysis.

Correctness. In Step 1, the sparse homomorphic decryption procedure outputs a BFV ciphertext encrypting

$$\mathbf{1}_r \otimes (\alpha\mu_0 + e_0, \dots, \alpha\mu_{m-1} + e_{m-1}).$$

In Step 2, by the definition of f_{poly} , we have $f_{\text{poly}}(\alpha\mu_i + e_i) = \alpha f(\mu_i)$ whenever $|e_i| < \lfloor q/(2p) \rfloor$. Hence Step 2 outputs a BFV ciphertext encrypting

$$\mathbf{1}_r \otimes (\alpha f(\mu_0), \dots, \alpha f(\mu_{m-1})).$$

Finally, Step 3 applies sparse SLOTTOCOEFF, SAMPLEEXTRACT, key switching, and modulus switching, which preserve the underlying plaintext. Therefore the output LWE ciphertexts encrypt $f(\mu_0), \dots, f(\mu_{m-1})$.

Noise analysis. Let σ be the standard deviation of the coefficient noise after Step 2 and the sparse SLOTTOCOEFF procedure. Since SAMPLEEXTRACT only selects coefficients and changes signs, it does not increase the noise standard deviation. The subsequent BFV key switching adds noise of standard deviation σ_{ks} . After modulus switching from Q' to q , the previous noise is scaled by q/Q' , and the final modulus-switching rounding contributes noise of standard deviation σ_{ms} . Thus the output LWE noise has standard deviation

$$\sigma_{\text{res}} = \sqrt{\left(\frac{q}{Q'}(\sigma + \sigma_{\text{ks}})\right)^2 + \sigma_{\text{ms}}^2}.$$

This gives the claimed output noise bound. $\square$

Complexity. Step 1 requires $\log(\min\{r, n\})$ homomorphic rotations and $\lceil n/r \rceil$ plaintext-ciphertext multiplications. Step 2 requires $2\log(d+1) - 2$ ciphertext-ciphertext multiplications, $\log(d+1) + m(d+1)/N$ plaintext-ciphertext multiplications, and $\tilde{O}(\sqrt{m(d+1)/N})$ rotations. Step 3 requires m plaintext-ciphertext multiplications and $O(\sqrt{m})$ rotations, together with the final key-switching and

modulus-switching steps. Since $d + 1$ is the smallest power of two satisfying $d \geq q - 1$ and $q = O(N)$, we have $d = O(N)$. Under the standing assumptions $N = O(\lambda)$ and $n = O(\lambda)$, the total number of BFV homomorphic operations is therefore $\tilde{O}(m)$. As each such operation can be implemented using $\tilde{O}(1)$ polynomial multiplications in R_Q , the total complexity is $\tilde{O}(m)$ polynomial multiplications in R_Q . In particular, when $m = 1$, the complexity is $\tilde{O}(1)$.

Failure probability. Assume that Q' is chosen so that $((q/Q')(\sigma + \sigma_{\text{ks}}))^2$ is negligible compared with the final modulus-switching noise. Following the random-fractional-part heuristic for non-randomized modulus switching of Ducas and Micciancio [18], the final rounding error is modeled as $e_{\text{ms}} = \eta_b - \sum_{i=0}^{n-1} s_i \eta_i$, where $\eta_b, \eta_0, \dots, \eta_{n-1}$ are independent centered uniform variables on $[-1/2, 1/2]$. For a dense sign LWE secret, we have $\|\mathbf{s}\|_2^2 = n$, and hence $\sigma_{\text{res}}^2 \approx \sigma_{\text{ms}}^2 \approx (n + 1)/12$. Let $B = \lfloor q/(2p) \rfloor$ be the LWE decryption threshold. Under this subgaussian model, the decryption-failure probability of one output ciphertext is bounded by

$$p_{\text{fail}}^{(1)} = \Pr[|e_{\text{out}}| \geq B] \leq 2 \exp\left(-\frac{6B^2}{n+1}\right).$$

6 Experiments

We implemented the functional bootstrapping algorithm described in Section 5 in Go on top of the Lattigo homomorphic encryption library (version 6.2.0), using its BFV interface, which in Lattigo v6 is implemented through the unified BFV/BGV integer-arithmetic backend. Our implementation supports arbitrary functions over the LWE plaintext space. In the following, we report experimental results for LWE plaintext moduli $p = 2^9, 2^{12}, 2^{14}, 2^{16}$. In all experiments, the LWE dimension is fixed to $n = 2048$, the BFV plaintext modulus T is set equal to the LWE ciphertext modulus q , and the LWE encoding factor is $\alpha = \lfloor q/p \rfloor$.

The input LWE ciphertexts have the form $b = \langle a, s \rangle + \alpha\mu + e \pmod{q}$. Unless an explicit secret vector is supplied, the LWE secret s is sampled coefficient-wise from the uniform ternary distribution on $\{-1, 0, 1\}$. The public LWE vector a is sampled uniformly from $\mathbb{Z}_q^n$. The input LWE error e is sampled as a rounded discrete Gaussian with standard deviation 3.2, rejection-sampled so that $|e| < \lfloor q/(2p) \rfloor$. For the underlying BFV/RLWE scheme, we do not override Lattigo's default secret and error distributions in the parameter literal. Thus the BFV/RLWE secret uses Lattigo's default ternary distribution, and the BFV/RLWE error uses Lattigo's default discrete Gaussian distribution, with standard deviation 3.2 and truncation bound 19.2. All parameter sets were chosen to provide at least 128-bit security, following the parameter selection guidelines in [6].

All experiments were conducted on a Windows desktop machine running Microsoft Windows 11 Home, version 10.0.22631, build 22631. The machine was equipped with an Intel(R) Core(TM) i9-14900K processor with 24 physical cores and 32 logical processors, and 31.69 GiB of RAM. The experiments were run from

Windows PowerShell 5.1.22621.6133 using Go 1.26.3 for windows/amd64. Unless otherwise stated, all reported timings are online times for one invocation of the corresponding bootstrapping procedure, averaged over 100 independent runs. No application-level parallelization across independent bootstrapping instances was used. To support reproducibility, the implementation and experimental scripts accompanying this paper are open source.⁴

The main comparison results are summarized in Table 2. The concrete parameter sets are listed in Table 6, and Table 7 gives the runtime breakdown, including the main online steps, key sizes, output-noise standard deviation, and analytic per-output failure-probability bounds.

Table 5. Parameters used for the functional bootstrapping experiments

Parameter set	N	$\log(PQ)$	d	q	p	$\log(Q)$				$\log(P)$
						Packing	PolyEval	StC	Base	
9-bit ($m = 1$)	32768	808	2^{16}	65537	2^9	30	34×18	0	29	34×4
9-bit ($m = 128$)	32768	845	2^{16}	65537	2^9	30	34×18	37	29	34×4
12-bit ($m = 1$)	65536	1100	$2^{20} - 1$	786433	2^{12}	38	38×22	0	36	38×5
12-bit ($m = 64$)	65536	1139	$2^{20} - 1$	786433	2^{12}	38	38×22	38	36	38×5
14-bit ($m = 1$)	65536	1236	$2^{22} - 1$	2752513	2^{14}	40	40×24	0	36	40×5
14-bit ($m = 32$)	65536	1276	$2^{22} - 1$	2752513	2^{14}	40	40×24	40	36	40×5
16-bit ($m = 1$)	65536	1443	$2^{23} - 1$	8257537	2^{16}	40	45×25	0	38	40×6
16-bit ($m = 16$)	65536	1488	$2^{23} - 1$	8257537	2^{16}	40	45×25	45	38	40×6

* The remaining **Base** modulus is the modulus level used for the final RLWE-to-LWE key switching.

** The entries in $\log(Q)$ and $\log(P)$ describe the bit-size pattern of the RNS primes used for the BFV ciphertext modulus and the auxiliary special modulus, respectively. For example, $45, 43 \times 24$ means one 45-bit prime and 24 primes of 43 bits, while 40×6 means six 40-bit primes. The column $\log(PQ)$ reports the bit length of the full RNS modulus product PQ used by the implementation. In all experiments, the LWE dimension is $n = 2048$, and the LWE secret is a sparse ternary vector with Hamming weight $h = 1024$.

[†] For $d = 2^{16}$, we first apply Algorithm 5 to the degree- $(d - 1)$ part, for which $(d - 1) + 1 = 2^{16}$, and then add the remaining term $a_d x^d$.

[‡] For the single-ciphertext rows with $m = 1$, the sparse **SlotToCoeff** transform is skipped, so the displayed StC budget is zero.

⁴ Available at <https://github.com/xiaopeng-stu/bfv-functional-bootstrapping>.

Table 6. Parameters used for the functional bootstrapping experiments

Parameter set	N	$\log(PQ)$	d	q	p	$\log(Q)$				$\log(P)$
						Packing	PolyEval	StC	Base	
9-bit ($m = 1$)	32768	902	2^{16}	65537	2^9	30	40×18	0	52	50×2
9-bit ($m = 128$)	32768	868	2^{16}	65537	2^9	30	37×18	37	45	45×2
12-bit ($m = 1, 64$)	65536	1660	$2^{20} - 1$	786433	2^{12}	30	52×23	52	52	55×6
14-bit ($m = 1, 32$)	65536	1712	$2^{22} - 1$	2752513	2^{14}	30	52×24	52	52	55×6
16-bit ($m = 1$)	65536	1717	$2^{23} - 1$	8257537	2^{16}	30	52×25	59	53	55×5
16-bit ($m = 16$)	65536	1723	$2^{23} - 1$	8257537	2^{16}	30	$59, 52 \times 24$	59	52	55×5

* Q is the initial BFV ciphertext modulus; the four entries under $\log(Q)$ give the budgets for packing, PolyEval, StC, and the remaining Base modulus, which is the modulu level used for the final RLWE-to-LWE key switching.

** The entries in $\log(Q)$ and $\log(P)$ list the bit-sizes of the RNS primes used in the implementation. Thus, for example, $52 \times 25, 30$ means 25 primes of 52 bits and one prime of 30 bits. The value $\log(PQ)$ is the total bit budget obtained from these listed prime sizes. In all experiments, the LWE dimension is $n = 2048$.

† For $d = 2^{16}$, we first apply Algorithm 5 to the degree- $(d-1)$ part, for which $(d-1)+1 = 2^{16}$, and then add the remaining term $a_d x^d$.

‡ For the single-ciphertext rows with $m = 1$, the sparse SlotToCoeff transform is skipped. If a nonzero modulus budget is displayed under StC, that budget is used instead for an additional modulus switch after polynomial evaluation to control the accumulated noise.

Table 7. Runtime breakdown of functional bootstrapping (seconds)

m	p	Bootstrap key (GiB)	Step 1	Step 2			Step 3	Online total	Noise σ_{res}	Per-LWE fail. prob.
				EvalPower	BatchLT	Total				
1	512	0.73	0.698	1.165	0.097	1.279	0.061	2.040	9.6344	$\leq 2^{-33.59}$
128	512	0.86	0.863	1.281	0.425	1.771	0.361	3.054	9.1499	
1	4096	1.79	1.854	3.811	0.297	4.184	0.151	6.198	7.8981	$\leq 2^{-76.83}$
64	4096	2.11	1.872	4.087	2.587	6.864	0.534	9.733	9.3026	
1	16384	2.29	2.398	4.617	0.414	5.134	0.152	7.715	8.6191	$\leq 2^{-58.59}$
32	16384	2.82	2.414	5.221	4.553	9.979	0.390	13.711	9.3183	
1	65536	2.17	2.196	4.954	0.604	5.691	0.164	8.113	10.3135	$\leq 2^{-32.52}$
16	65536	2.90	2.282	5.386	4.073	9.696	0.308	13.220	9.6781	

^a Times are averages over 100 independent runs and refer to the online bootstrapping phase.

^b The bootstrap-key size includes the BFV encryptions of the LWE secret and all auxiliary keys. The noise column reports the empirical standard deviation of the output LWE noise over all output samples collected in the 100 runs.

^c The last column reports the secret-aware sparse analytic failure-probability bound for each output LWE ciphertext, using $\|s\|_2^2 + 1 = h + 1$.

^d Within Step 2, EvalPower and BatchLT are listed separately because they account for most of the homomorphic polynomial-evaluation cost.

7 Conclusion

In this paper, we gave a functional bootstrapping method for a single LWE ciphertext with non-amortized $\tilde{O}(1)$ polynomial multiplications, and more generally for small and moderate batch sizes. Our main technical ingredient is a new algorithm for batched homomorphic polynomial evaluation on sparsely packed BFV ciphertexts. Under the condition $d < (N/m)^2$ and $d = O(N)$, the proposed method evaluates m polynomials on m encrypted inputs using only $\tilde{O}(m)$ polynomial multiplications, which yields amortized $\tilde{O}(1)$ complexity per input. In particular, when $m = 1$, it gives a functional bootstrapping procedure for a single LWE ciphertext using only $\tilde{O}(1)$ polynomial multiplications, rather than only in an amortized sense.

Building on this ingredient and the BFV-based framework of [44], we obtained a large-plaintext functional bootstrapping scheme for arbitrary functions that is well suited to practical settings where only a moderate number of LWE ciphertexts need to be refreshed at a time. Our implementation in Lattigo shows that the resulting method supports plaintext spaces of 9, 12, 14, and 16 bits and achieves practical performance in both the single-ciphertext and small-batch settings. These results suggest that small-batch functional bootstrapping constitutes a practically relevant regime that is complementary to existing large-batch amortized approaches.

Acknowledgements. This research was supported by the National Key Research and Development Project under Grant No. 2025YFA1017200 and STU-Scientific Research Initiation Grant No. NTF24023T.

References

1. Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. General Functional Bootstrapping Using CKKS. In *Advances in Cryptology – CRYPTO 2025, Part III*, volume 16002 of *Lecture Notes in Computer Science*, pages 304–337. Springer Nature Switzerland, 2025.
2. Jacob Alperin-Sheriff and Chris Peikert. Practical Bootstrapping in Quasilinear Time. In *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 1–20. Springer, 2013.
3. Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Bootstrapping Small Integers with CKKS. In *Advances in Cryptology – ASIACRYPT 2024, Part I*, volume 15484 of *Lecture Notes in Computer Science*, pages 330–360. Springer Singapore, 2024.
4. Song Bian, Zhou Zhang, Haowen Pan, Ran Mao, Zian Zhao, Yier Jin, and Zhenyu Guan. HE³DB: An Efficient and Elastic Encrypted Database Via Arithmetic-And-Logic Fully Homomorphic Encryption. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 2930–2944. ACM, 2023.
5. Dan Boneh, Craig Gentry, Shai Halevi, Frank Wang, and David J. Wu. Private Database Queries Using Somewhat Homomorphic Encryption. In *Applied Cryptography and Network Security – ACNS 2013*, volume 7954 of *Lecture Notes in Computer Science*, pages 102–118. Springer, 2013.

6. Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R. Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, Changmin Lee, Xianhui Lu, Carsten Maple, Alberto Pedrouzo-Ulloa, Rachel Player, Yuriy Polyakov, Luis Antonio Ruiz Lopez, Yongsoo Song, and Donggeon Yhee. Security Guidelines for Implementing Homomorphic Encryption. *IACR Communications in Cryptology*, 1(4), 2025.
7. Jean-Philippe Bossuat, Christian Mouchet, Juan Ramón Troncoso-Pastoriza, and Jean-Pierre Hubaux. Efficient Bootstrapping for Approximate Homomorphic Encryption with Non-sparse Keys. In *Advances in Cryptology – EUROCRYPT 2021, Part I*, volume 12696 of *Lecture Notes in Computer Science*, pages 587–617. Springer, 2021.
8. Zvika Brakerski. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In *Advances in Cryptology – CRYPTO 2012*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886. Springer, 2012.
9. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) Fully Homomorphic Encryption without Bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, ITCS ’12, pages 309–325. ACM, 2012.
10. Hao Chen, Ilaria Chillotti, and Yongsoo Song. Improved Bootstrapping for Approximate Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT 2019, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 34–54. Springer, 2019.
11. Hao Chen and Kyoohyung Han. Homomorphic Lower Digits Removal and Improved FHE Bootstrapping. In *Advances in Cryptology – EUROCRYPT 2018, Part I*, volume 10820 of *Lecture Notes in Computer Science*, pages 315–337. Springer, 2018.
12. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for Approximate Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT 2018, Part I*, volume 10820 of *Lecture Notes in Computer Science*, pages 360–384. Springer, 2018.
13. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *Advances in Cryptology – ASIACRYPT 2017, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437. Springer, 2017.
14. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster Fully Homomorphic Encryption: Bootstrapping in Less Than 0.1 Seconds. In *Advances in Cryptology – ASIACRYPT 2016, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 3–33. Springer, 2016.
15. Ilaria Chillotti, Marc Joye, and Pascal Paillier. Programmable Bootstrapping Enables Efficient Homomorphic Inference of Deep Neural Networks. In *Cyber Security Cryptography and Machine Learning – CSCML 2021*, volume 12716 of *Lecture Notes in Computer Science*, pages 1–19. Springer, 2021.
16. Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. Improved Programmable Bootstrapping with Larger Precision and Efficient Arithmetic Circuits for TFHE. In *Advances in Cryptology – ASIACRYPT 2021, Part III*, volume 13092 of *Lecture Notes in Computer Science*, pages 670–699. Springer, 2021.
17. Kuiyuan Duan, Hongbo Li, Dengfa Liu, and Guangsheng Ma. Large-Plaintext Functional Bootstrapping in FHE with Small Bootstrapping Keys. In *Theory of Cryptography – TCC 2025, Part I*, volume 16268 of *Lecture Notes in Computer Science*, pages 253–279. Springer Nature Switzerland, 2025.

18. Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In *Advances in Cryptology – EUROCRYPT 2015, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 617–640. Springer, 2015.
19. Junfeng Fan and Frederik Vercauteren. Somewhat Practical Fully Homomorphic Encryption. *IACR Cryptology ePrint Archive*, 2012:144, 2012.
20. Robin Geelen, Iliia Iliashenko, Jiayi Kang, and Frederik Vercauteren. On Polynomial Functions Modulo p^e and Faster Bootstrapping for Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 257–286. Springer Nature Switzerland, 2023.
21. Robin Geelen and Frederik Vercauteren. Bootstrapping for BGV and BFV Revisited. *Journal of Cryptology*, 36(2):12, 2023.
22. Craig Gentry. *A Fully Homomorphic Encryption Scheme*. PhD thesis, Stanford University, 2009.
23. Craig Gentry, Shai Halevi, and Nigel P. Smart. Better Bootstrapping in Fully Homomorphic Encryption. In *Public-Key Cryptography – PKC 2012*, volume 7293 of *Lecture Notes in Computer Science*, pages 1–16. Springer, 2012.
24. Craig Gentry, Shai Halevi, and Nigel P. Smart. Fully Homomorphic Encryption with Polylog Overhead. In *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 465–482. Springer, 2012.
25. Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In *Advances in Cryptology – CRYPTO 2013, Part I*, volume 8042 of *Lecture Notes in Computer Science*, pages 75–92. Springer, 2013.
26. Ran Gilad-Bachrach, Nathan Dowlin, Kim Laine, Kristin Lauter, Michael Naehrig, and John Wernsing. CryptoNets: Applying Neural Networks to Encrypted Data with High Throughput and Accuracy. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 201–210. PMLR, 2016.
27. Antonio Guimarães, Edson Borin, and Diego F. Aranha. Revisiting the Functional Bootstrap in TFHE. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(2):229–253, 2021.
28. Antonio Guimarães and Hilder V. L. Pereira. Fast amortized bootstrapping with small keys and polynomial noise overhead. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS ’25)*, pages 2967–2981, New York, NY, USA, 2025. Association for Computing Machinery.
29. Antonio Guimarães, Hilder V. L. Pereira, and Barry van Leeuwen. Amortized Bootstrapping Revisited: Simpler, Asymptotically-Faster, Implemented. In *Advances in Cryptology – ASIACRYPT 2023, Part VI*, volume 14443 of *Lecture Notes in Computer Science*, pages 3–35. Springer Nature Singapore, 2023.
30. Shai Halevi and Victor Shoup. Algorithms in HELib. In Juan A. Garay and Rosario Gennaro, editors, *Advances in Cryptology – CRYPTO 2014, Part I*, volume 8616 of *Lecture Notes in Computer Science*, pages 554–571. Springer, Berlin, Heidelberg, 2014.
31. Shai Halevi and Victor Shoup. Bootstrapping for HELib. In *Advances in Cryptology – EUROCRYPT 2015, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 641–670. Springer, 2015.
32. Chiraag Juvekar, Vinod Vaikuntanathan, and Anantha Chandrakasan. GAZELLE: A Low Latency Framework for Secure Neural Network Inference. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 1651–1669. USENIX Association, 2018.

33. Andrey Kim, Yuriy Polyakov, and Vincent Zucca. Revisiting Homomorphic Encryption Schemes for Finite Fields. In *Advances in Cryptology – ASIACRYPT 2021, Part III*, volume 13092 of *Lecture Notes in Computer Science*, pages 608–639. Springer, 2021.
34. Duhyeon Kim and Yongsoo Song. Approximate homomorphic encryption over the conjugate-invariant ring. In Kwangsu Lee, editor, *Information Security and Cryptology – ICISC 2018*, volume 11396 of *Lecture Notes in Computer Science*, pages 85–102. Springer, Cham, 2019.
35. Kamil Kluczniak and Leonard Schild. FDFB: Full-Domain Functional Bootstrapping Towards Practical Fully Homomorphic Encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2023(1):501–537, 2023.
36. Lattigo v6.2.0. Online: <https://github.com/tuneinsight/lattigo>, February 2026. EPFL-LDS, Tune Insight SA.
37. Dongwon Lee, Seonhong Min, and Yongsoo Song. Functional Bootstrapping for Packed Ciphertexts via Homomorphic LUT Evaluation. *Cryptology ePrint Archive*, Paper 2024/181, 2024.
38. Yongwoo Lee, Joon-Woo Lee, Young-Sik Kim, Yongjune Kim, Jong-Seon No, and HyungChul Kang. High-Precision Bootstrapping for Approximate Homomorphic Encryption by Error Variance Minimization. In *Advances in Cryptology – EUROCRYPT 2022, Part I*, volume 13275 of *Lecture Notes in Computer Science*, pages 551–580. Springer, 2022.
39. Yongwoo Lee, Daniele Micciancio, Andrey Kim, Rakyong Choi, Maxim Deryabin, Jieun Eom, and Donghoon Yoo. Efficient FHEW Bootstrapping with Small Evaluation Keys, and Applications to Threshold Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 227–256. Springer Nature Switzerland, 2023.
40. Zhihao Li, Benqiang Wei, Ruida Wang, Xianhui Lu, and Kunpeng Wang. Full Domain Functional Bootstrapping with Least Significant Bit Encoding. In *Information Security and Cryptology – Inscrypt 2023, Part I*, volume 14526 of *Lecture Notes in Computer Science*, pages 203–223. Springer Nature Singapore, 2024.
41. Feng-Hao Liu and Han Wang. Batch Bootstrapping I: A New Framework for SIMD Bootstrapping in Polynomial Modulus. In *Advances in Cryptology – EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 321–352. Springer Nature Switzerland, 2023.
42. Feng-Hao Liu and Han Wang. Batch Bootstrapping II: Bootstrapping in Polynomial Modulus Only Requires $\tilde{O}(1)$ FHE Multiplications in Amortization. In *Advances in Cryptology – EUROCRYPT 2023, Part III*, volume 14006 of *Lecture Notes in Computer Science*, pages 353–384. Springer Nature Switzerland, 2023.
43. Zeyu Liu, Daniele Micciancio, and Yuriy Polyakov. Large-Precision Homomorphic Sign Evaluation Using FHEW/TFHE Bootstrapping. In *Advances in Cryptology – ASIACRYPT 2022, Part II*, volume 13792 of *Lecture Notes in Computer Science*, pages 130–160. Springer Nature Switzerland, 2022.
44. Zeyu Liu and Yunhao Wang. Amortized Functional Bootstrapping in Less than 7 ms, with $\tilde{O}(1)$ Polynomial Multiplications. In *Advances in Cryptology – ASIACRYPT 2023, Part VI*, volume 14443 of *Lecture Notes in Computer Science*, pages 101–132. Springer Nature Singapore, 2023.
45. Wen-jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. PEGASUS: Bridging Polynomial and Non-polynomial Evaluations in Homomorphic Encryption. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1057–1073. IEEE, 2021.

46. Shihe Ma, Tairong Huang, Anyu Wang, Qixian Zhou, and Xiaoyun Wang. Fast and Accurate: Efficient Full-Domain Functional Bootstrap and Digit Decomposition for Homomorphic Computation. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(1):592–616, 2024.
47. Daniele Micciancio and Jessica Sorrell. Ring Packing and Amortized FHEW Bootstrapping. In *45th International Colloquium on Automata, Languages, and Programming (ICALP 2018)*, volume 107 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 100:1–100:14. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018.
48. Gabrielle De Micheli, Duhyeong Kim, Daniele Micciancio, and Adam Suhl. Faster Amortized FHEW Bootstrapping Using Ring Automorphisms. In *Public-Key Cryptography – PKC 2024, Part IV*, volume 14604 of *Lecture Notes in Computer Science*, pages 322–353. Springer Nature Switzerland, 2024.
49. Hiroki Okada, Rachel Player, and Simon Pohmann. Homomorphic Polynomial Evaluation Using Galois Structure and Applications to BFV Bootstrapping. In *Advances in Cryptology – ASIACRYPT 2023, Part VI*, volume 14443 of *Lecture Notes in Computer Science*, pages 69–100. Springer, 2023.
50. Jaiyoung Park, Sejin Park, Jai Hyun Park, Jung Ho Ahn, Jung Hee Cheon, Guillaume Hanrot, Jung Woo Kim, Minje Park, and Damien Stehlé. Scaling up Privacy-Preserving ML: A CKKS Implementation of Llama-2-7B. *arXiv preprint arXiv:2601.18511*, 2026.
51. Michael S. Paterson and Larry J. Stockmeyer. On the Number of Nonscalar Multiplications Necessary to Evaluate Polynomials. *SIAM Journal on Computing*, 2(1):60–66, 1973.
52. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the 37th Annual ACM Symposium on Theory of Computing*, pages 84–93. ACM, 2005.
53. Qingfeng Wang and Li-Ping Wang. A Novel Asymmetric BSGS Polynomial Evaluation Algorithm under Homomorphic Encryption. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, pages 30–44. ACM, 2025.
54. Ruida Wang, Xianhui Lu, Yundi Wen, Zhihao Li, Benqiang Wei, Kunpeng Wang, and Lixia Luo. Full Domain Functional Bootstrapping Using the Prime Cyclotomic Ring. *Theoretical Computer Science*, 1050:115392, 2025.
55. Han Xia, Feng-Hao Liu, and Han Wang. More Efficient Functional Bootstrapping for General Functions in Polynomial Modulus. In *Theory of Cryptography – TCC 2024, Part IV*, volume 15367 of *Lecture Notes in Computer Science*, pages 130–163. Springer Nature Switzerland, 2024.
56. Yan Xu, Guizhen Zhu, Huaxiong Wang, and Li-Ping Wang. Large-Plaintext Functional Bootstrapping with Small Parameters for BGV Encoding. In *Information Security – ISC 2024, Part I*, volume 15257 of *Lecture Notes in Computer Science*, pages 252–272. Springer Nature Switzerland, 2024.